



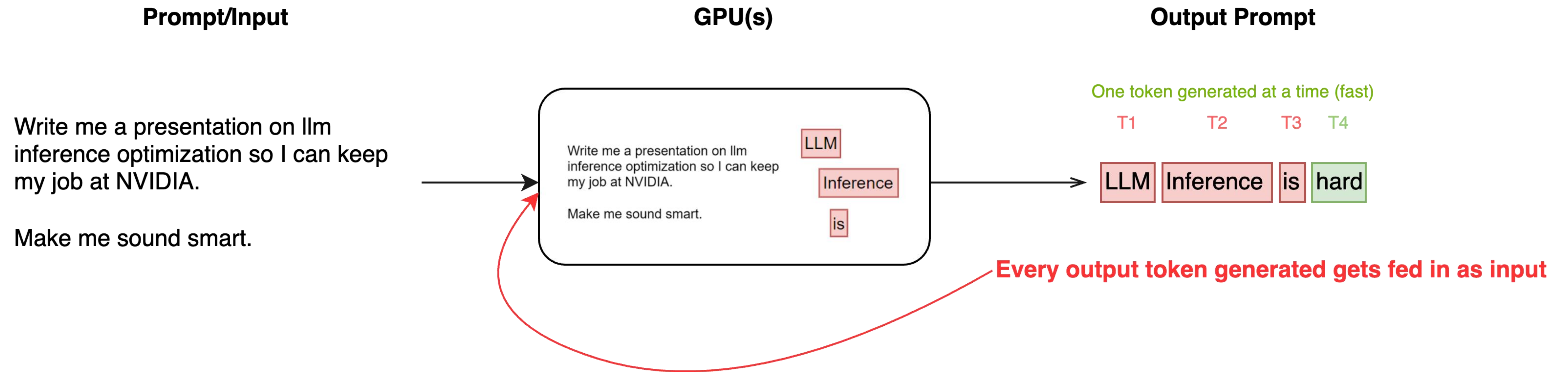
Understanding the LLM Inference Workload

Mark Moyou, PhD - Sr. Data Scientist/Solutions Architect

Understanding LLM Inference

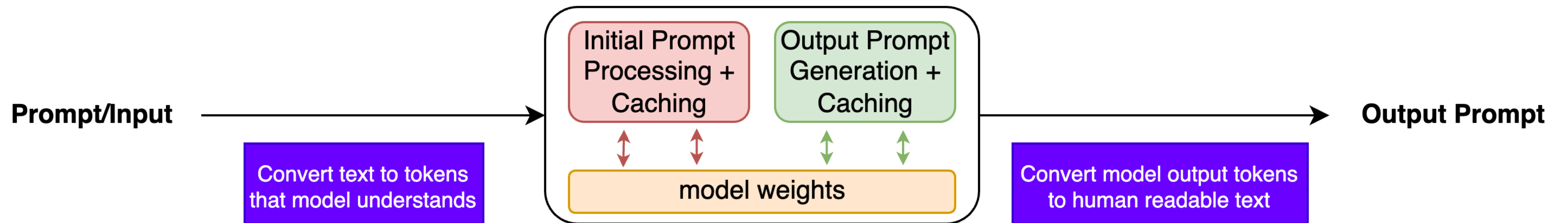
Understanding LLM Inference

Input and output flows



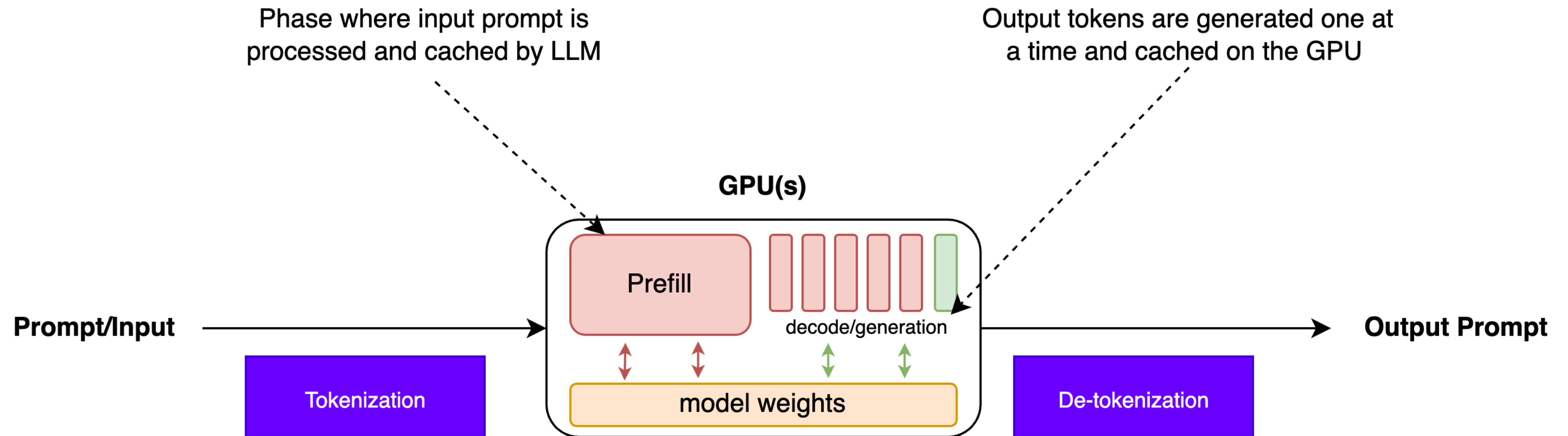
Understanding LLM Inference

Main processing blocks – Intuitive Version



Understanding LLM Inference

End to end with technical terms



How Text Data is Transformed for LLM Processing

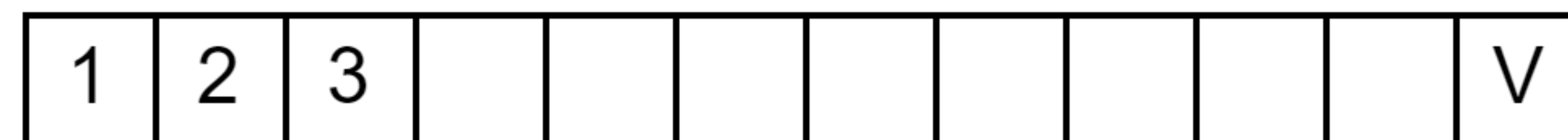
Tokenization – Different LLMs have Different Tokenizers

Tokenization - Converting the input sentence/sequence into a sequence of LLM token ids

token $\sim$ 4 characters



← Total vocabulary of tokens for the LLM



← Token ids corresponding to tokens

Convert input text /prompt



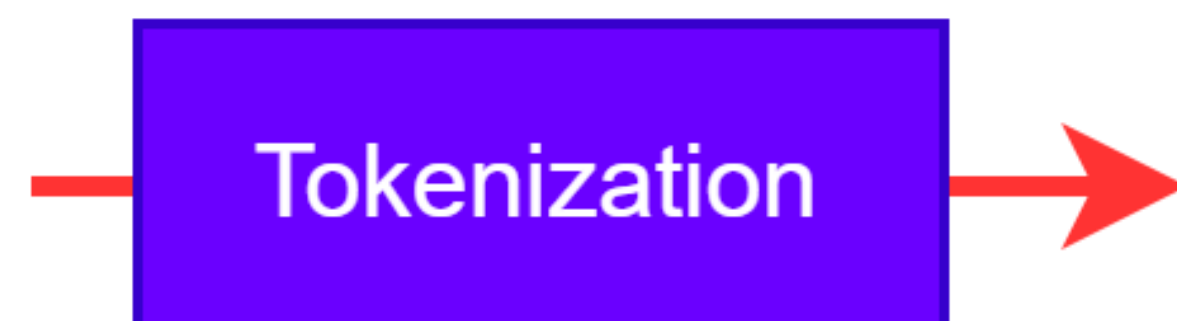
Tokenization



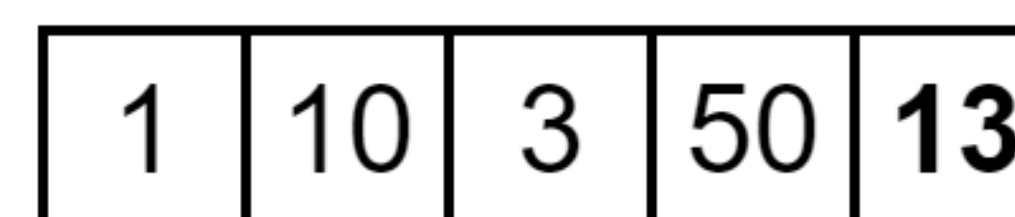
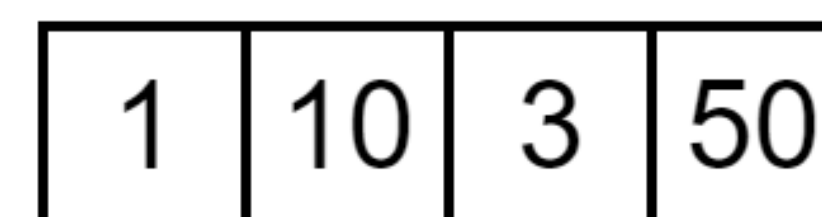
Vector of token ids

Make me sound smart

Make me sound smarter



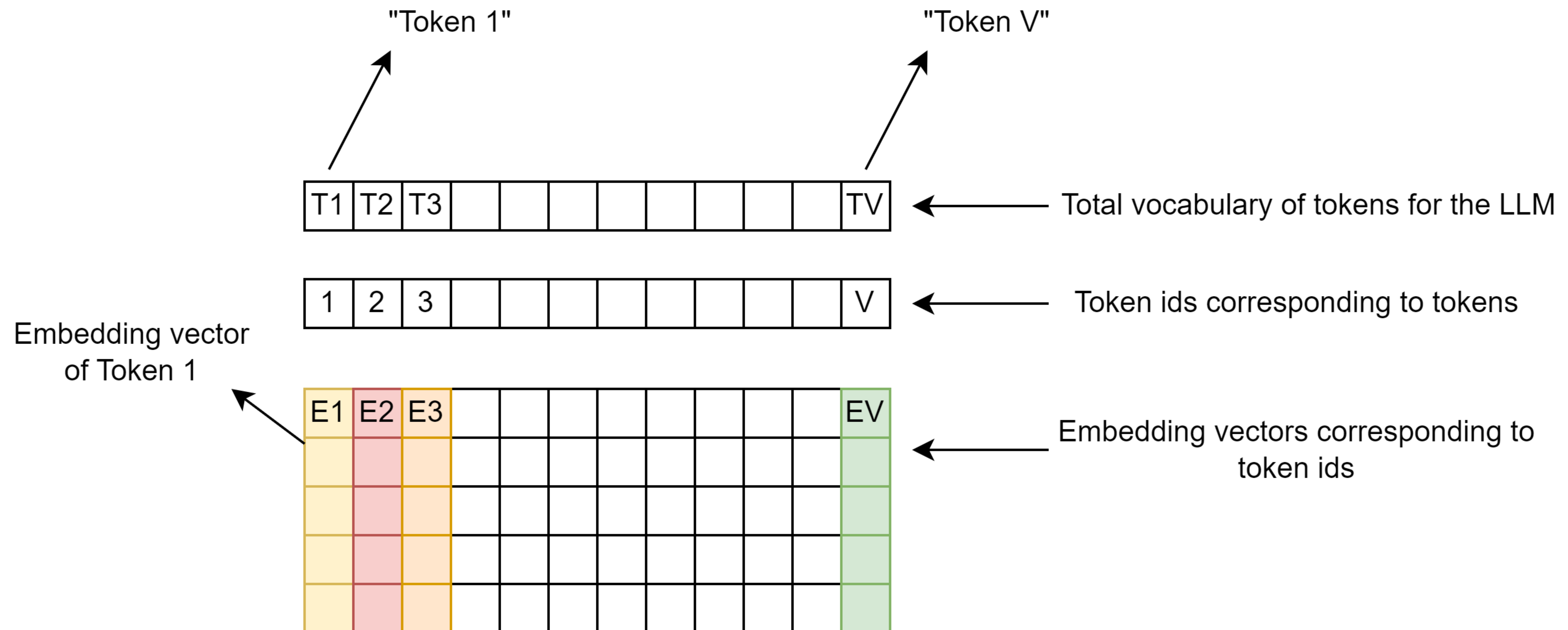
Tokenization



Vector representation of text

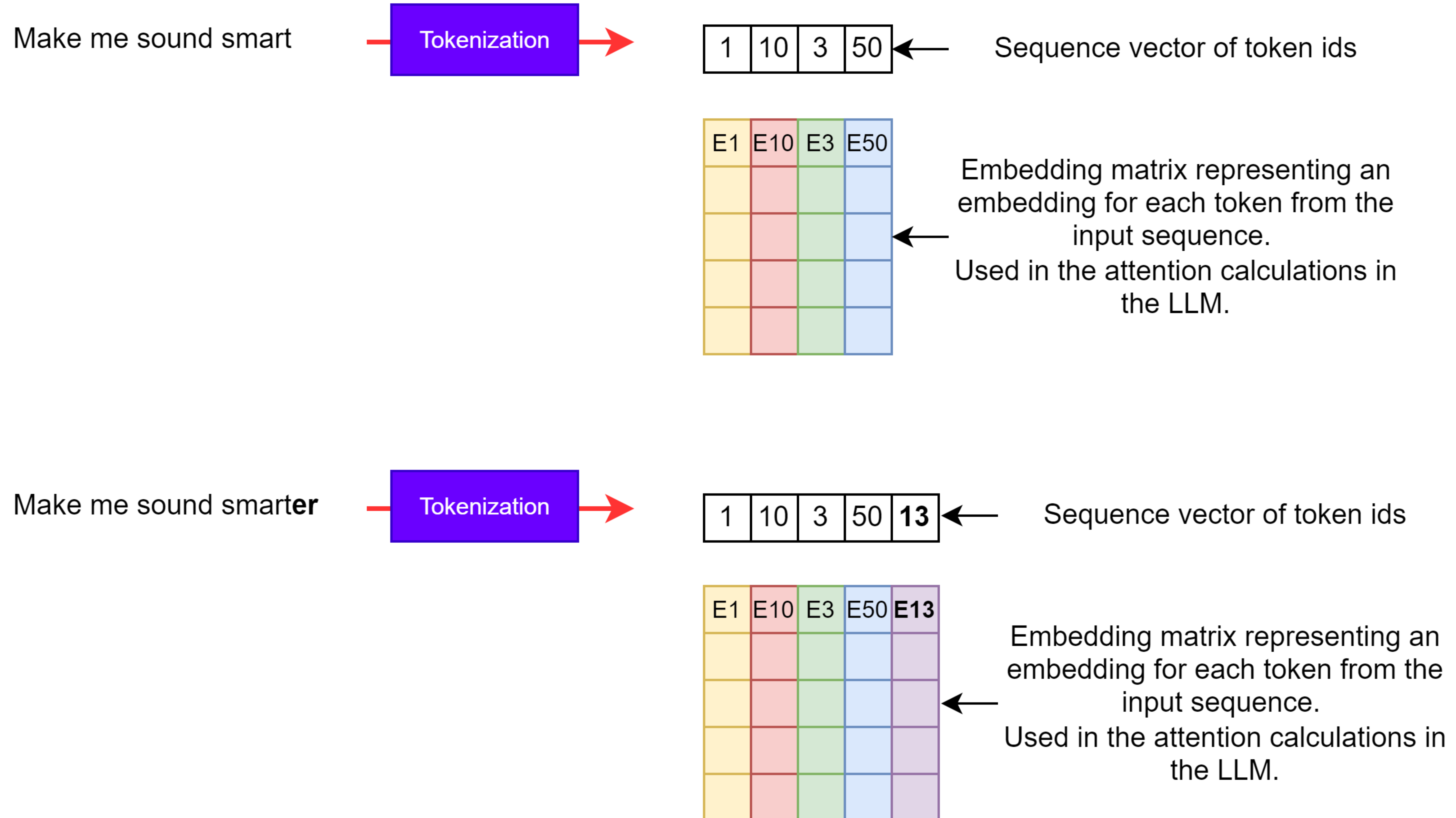
Every LLM Token Has An Embedding/Vector

A sequence of human words is really a sequence of vectors aka a matrix



Matrix Representation of an Input Prompt

Tokenization - Converting the input sentence/sequence into a sequence of LLM token ids



An input prompt to an LLM is stored as a matrix on the GPU

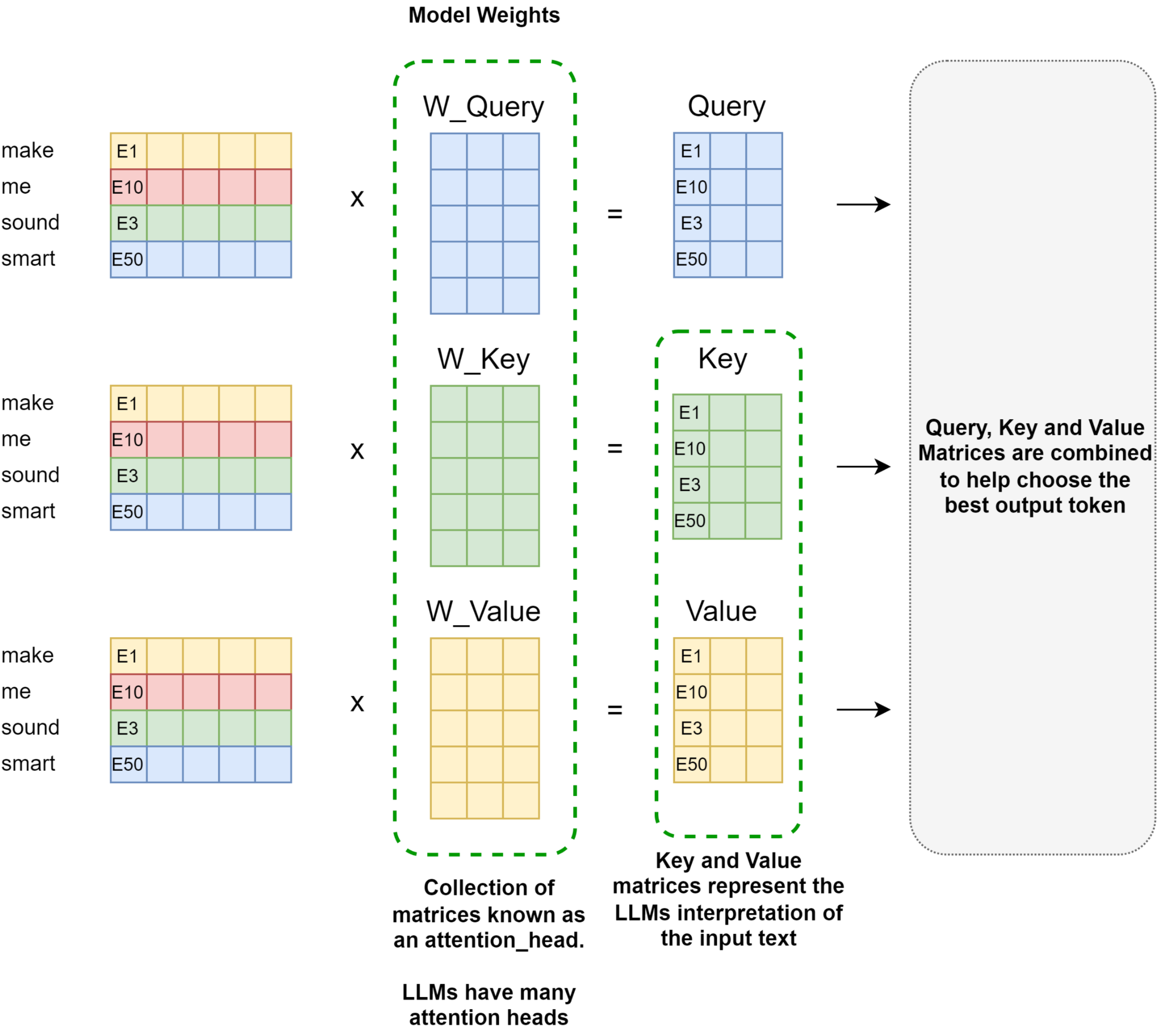
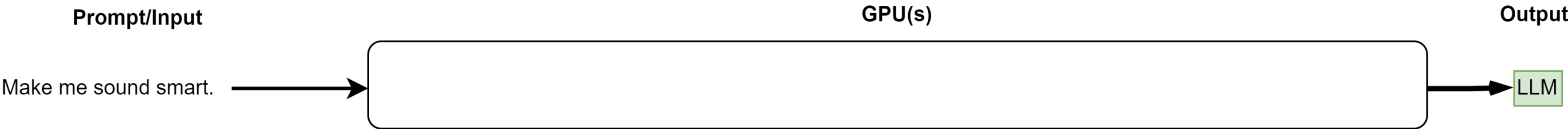
The larger the prompt the **larger the matrix on the GPU**

How LLMs Processes Tokens

Attention Mechanism enables LLMs to focus on important information (tokens)

How LLMs Make Sense Of Tokens Using Attention

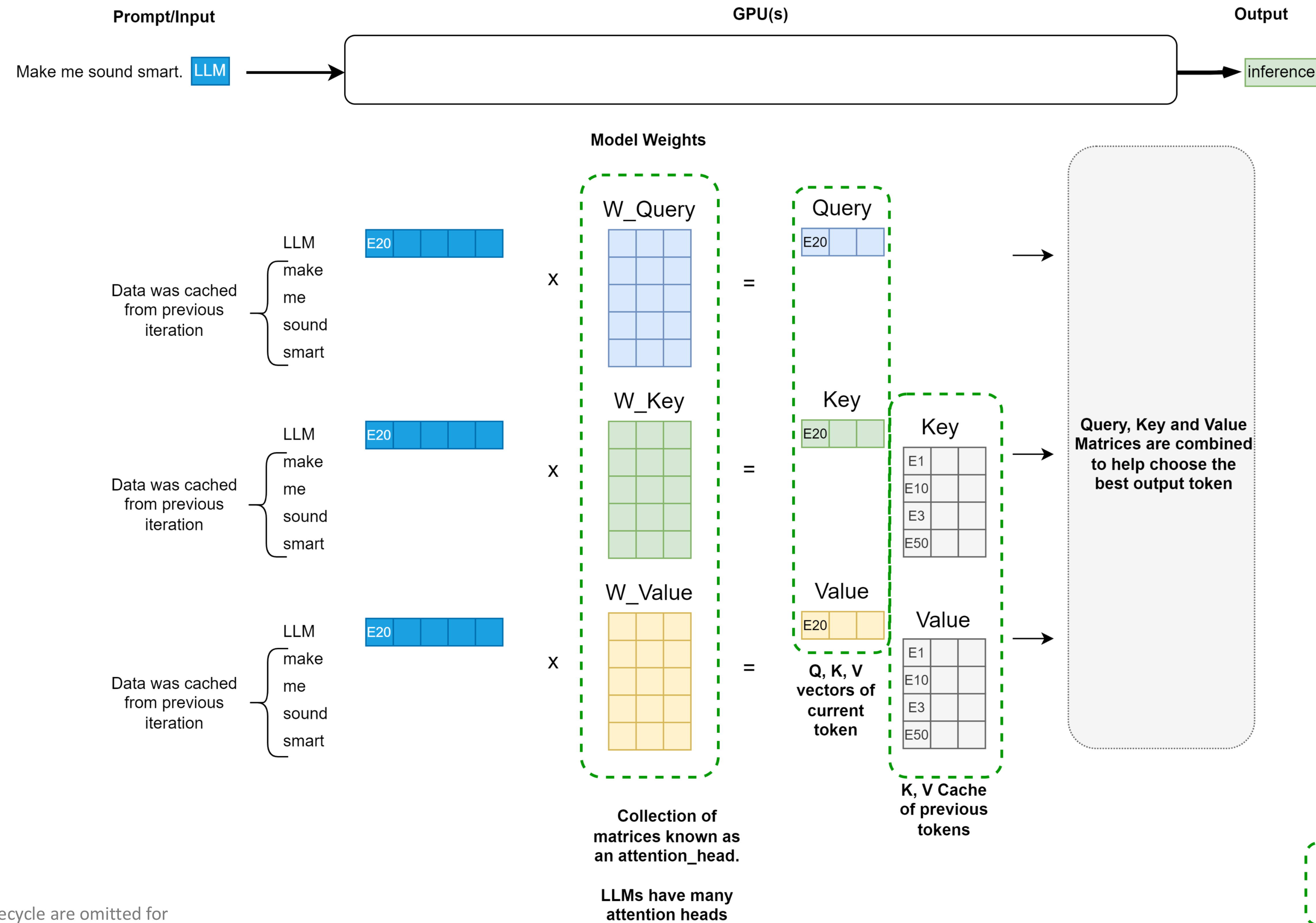
Convert text to token-ids -> token embeddings -> compute attention on the entire input sequence



Many details of the model inference lifecycle are omitted for brevity such as mlps, layer-norms, decoding, etc.

The Value Of The Key-Value (KV) Cache – Increase Inference Speed

KV Cache allows new tokens to be generated much faster

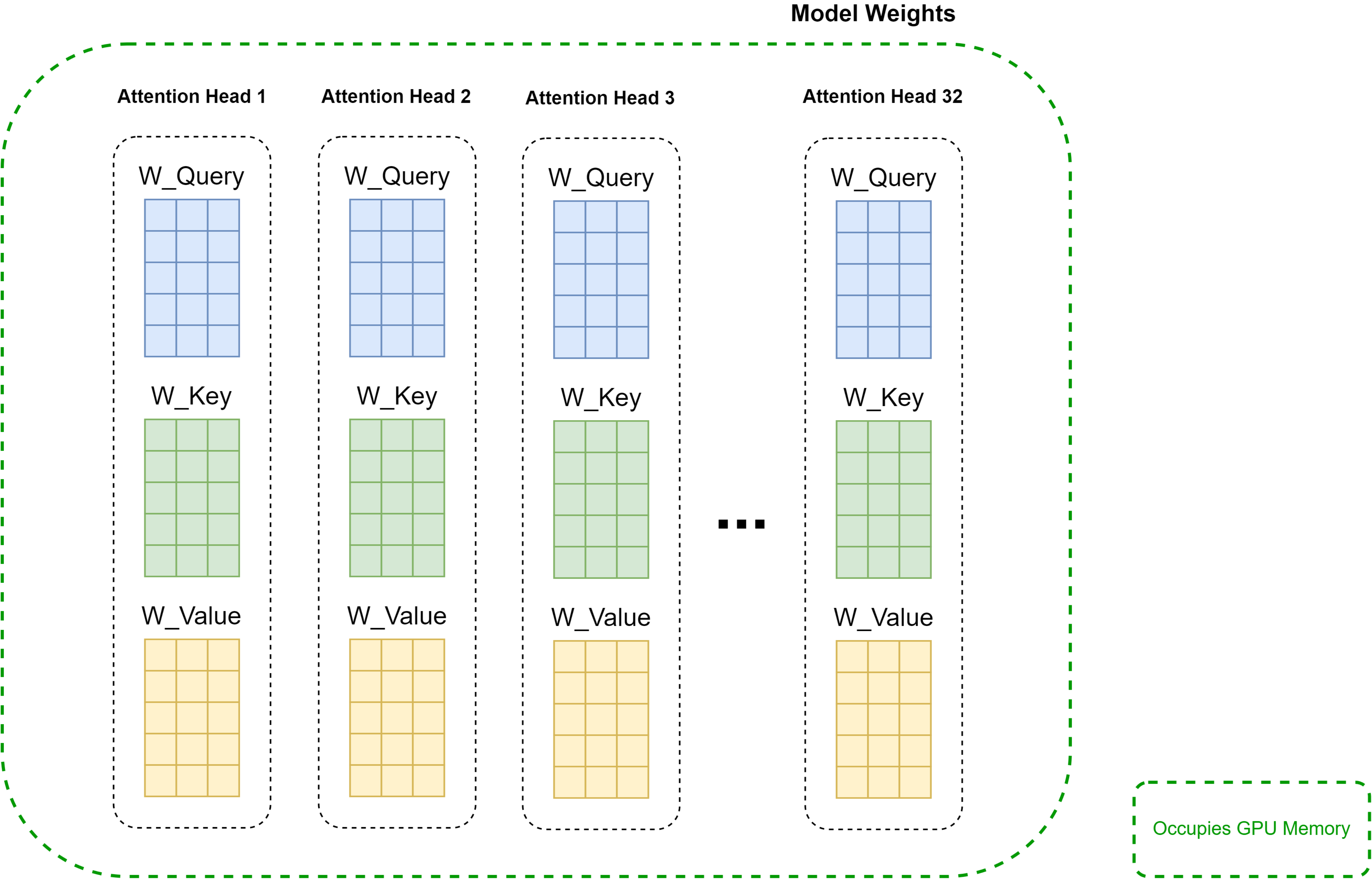


LLM Parameters – Multiple Attention Heads

Each attention head learns different information

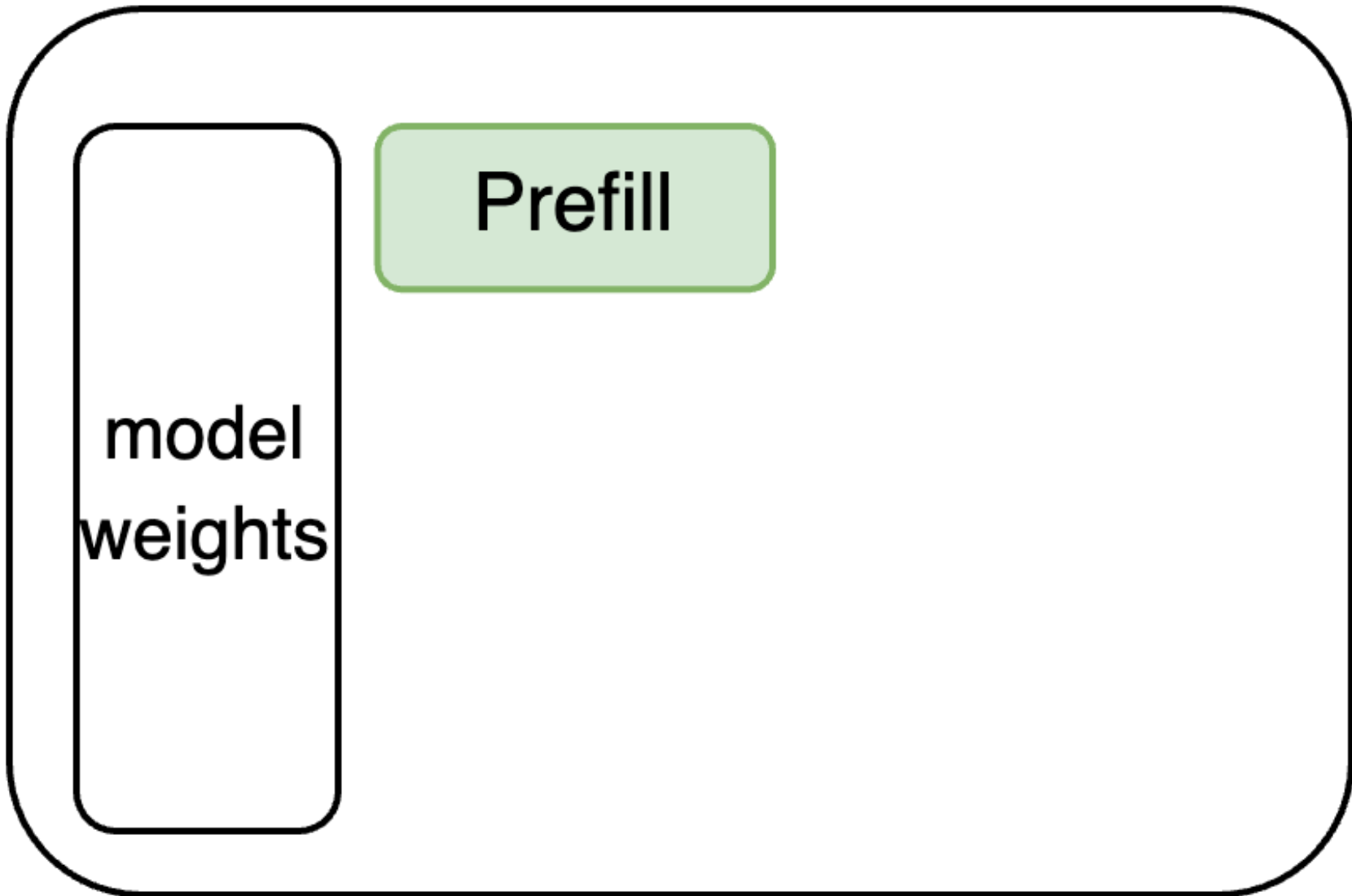


Llama3 has 32 attention heads



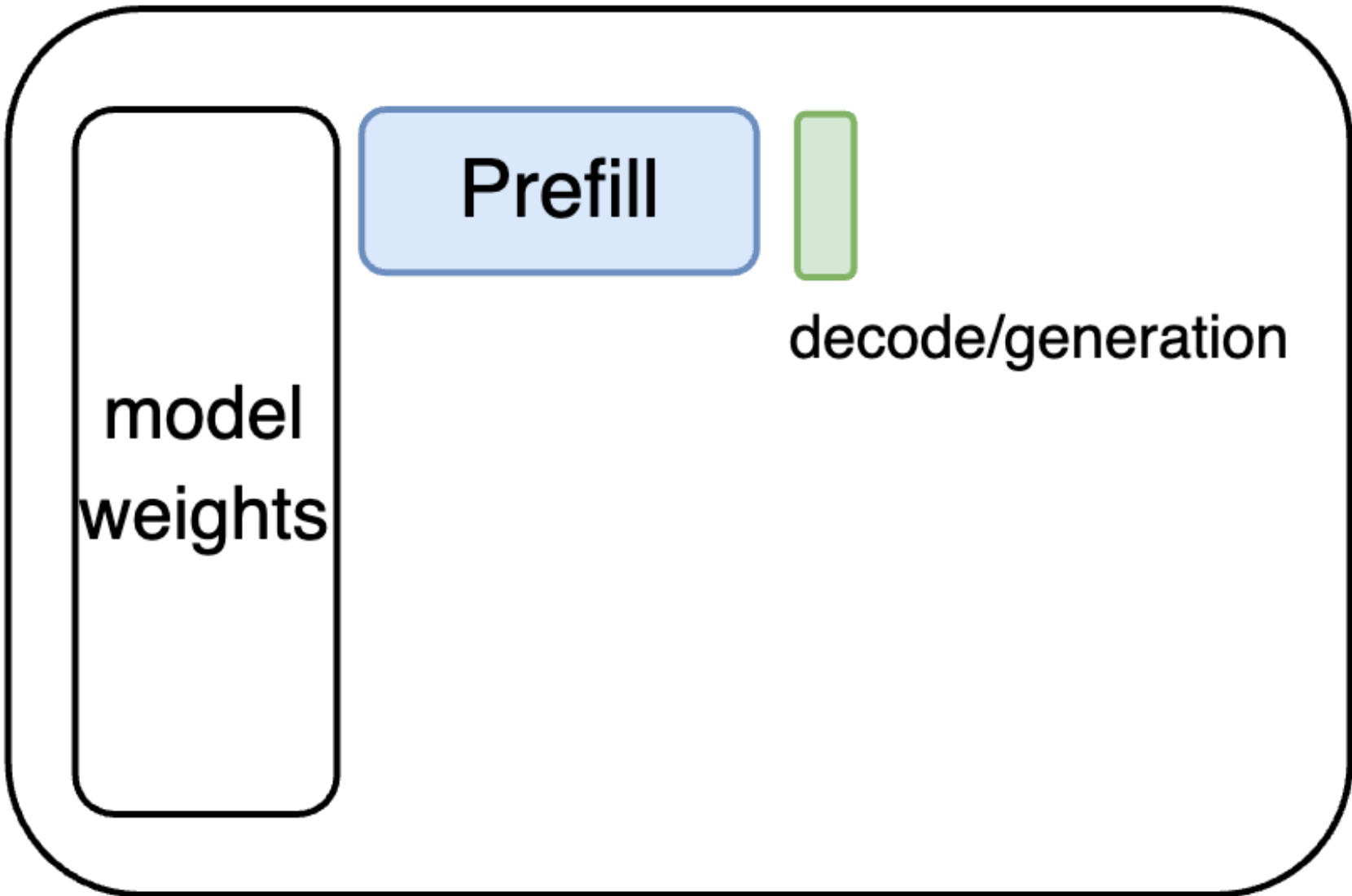
Inference Loop

**Forward Pass
Prefill Only**



Prompt is processed in prefill stage to compute the attention mechanism on the entire prompt (single request case)

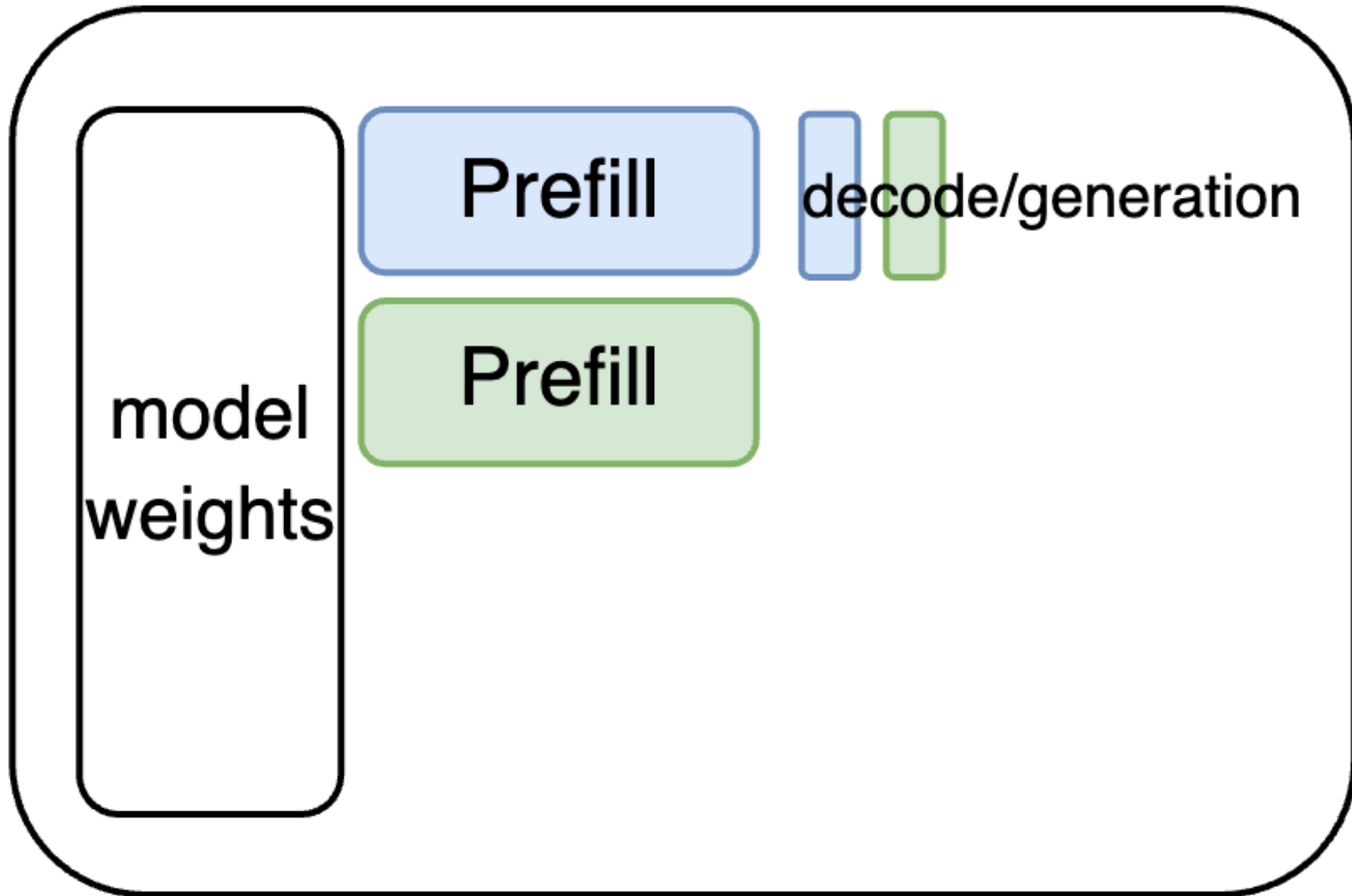
**Forward Pass
Generation Only**



Prefill is finished and first token is generated for the current request (single request case)

In this phase ~95% of the time

**Forward Pass
Prefill + Generation**



New request comes in, needs to do prefill while 2nd generation from request 1 is in process.

Doing prefill and generation is called piggybacking

Efficient prefill strategy is needed

Legend

Completed stage

Stage currently processing

Increasing Throughput with Chunked Prefill

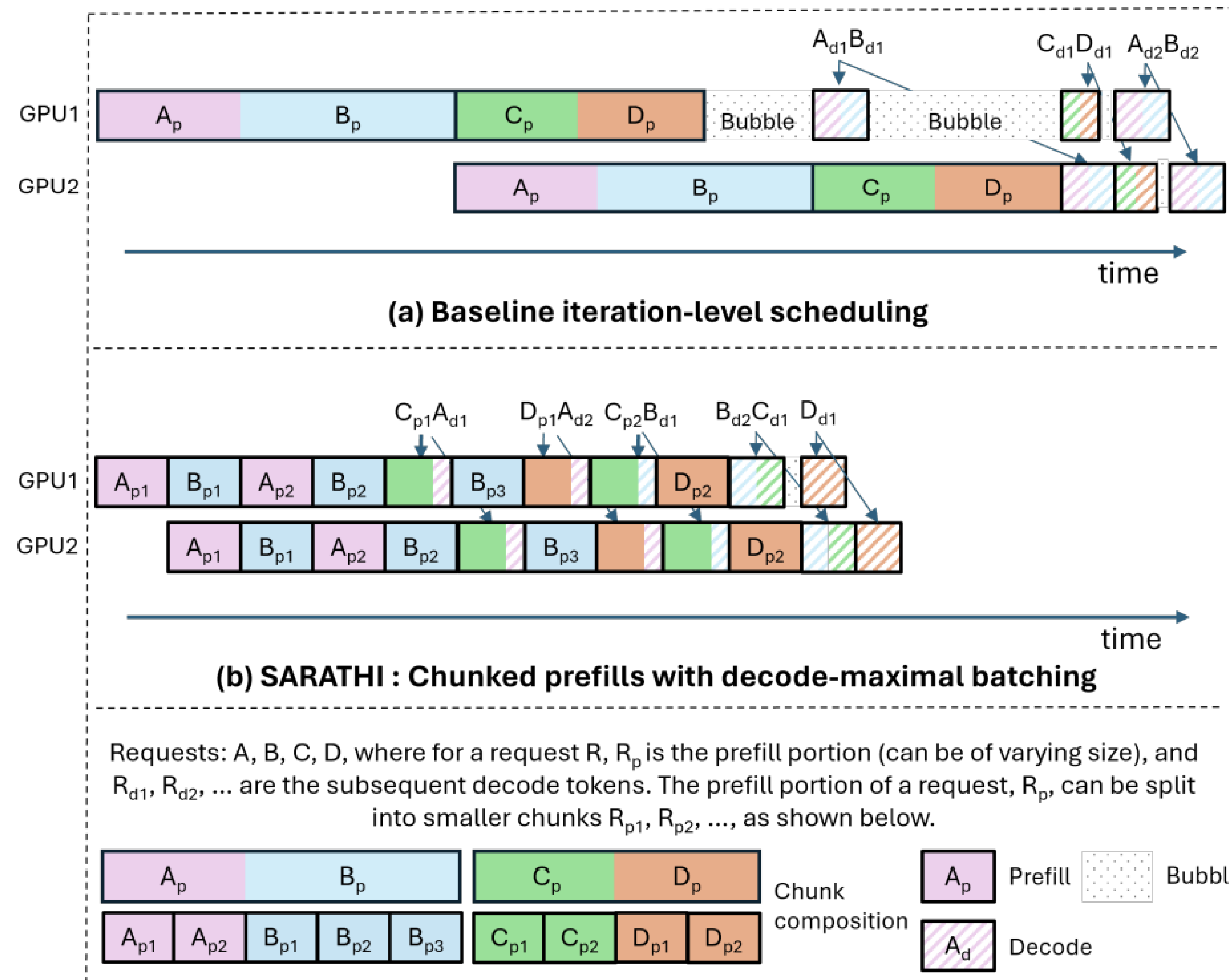
SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills

Amey Agrawal^{*2}, Ashish Panwar¹, Jayashree Mohan¹, Nipun Kwatra¹, Bhargav S. Gulavani¹, and
Ramachandran Ramjee¹

¹Microsoft Research India

²Georgia Institute of Technology

Increasing Throughput with Chunked Prefill – More Efficient Decoding



Size of model on GPU (GB) in FP16 = #parameters x 2

Llama 3-8B (FP16) is 8 Billion parameters x 2 = 16GB

Llama 3-8B (FP8) is 8 Billion parameters x 1 = 8GB

KV Cache Sizing

This memory size is required for each token in the input sequence, across the batch of inputs. Assuming half-precision, the total size of KV cache is given by the formula below.

Total size of KV cache in bytes = (batch_size) * (sequence_length) * 2 * (num_layers) * (hidden_size) * sizeof(FP16)

For example, with a Llama 2 7B model in 16-bit precision and a batch size of 1, the size of the KV cache will be $1 * 4096 * 2 * 32 * 4096 * 2$ bytes, which is ~2 GB.

BLOG

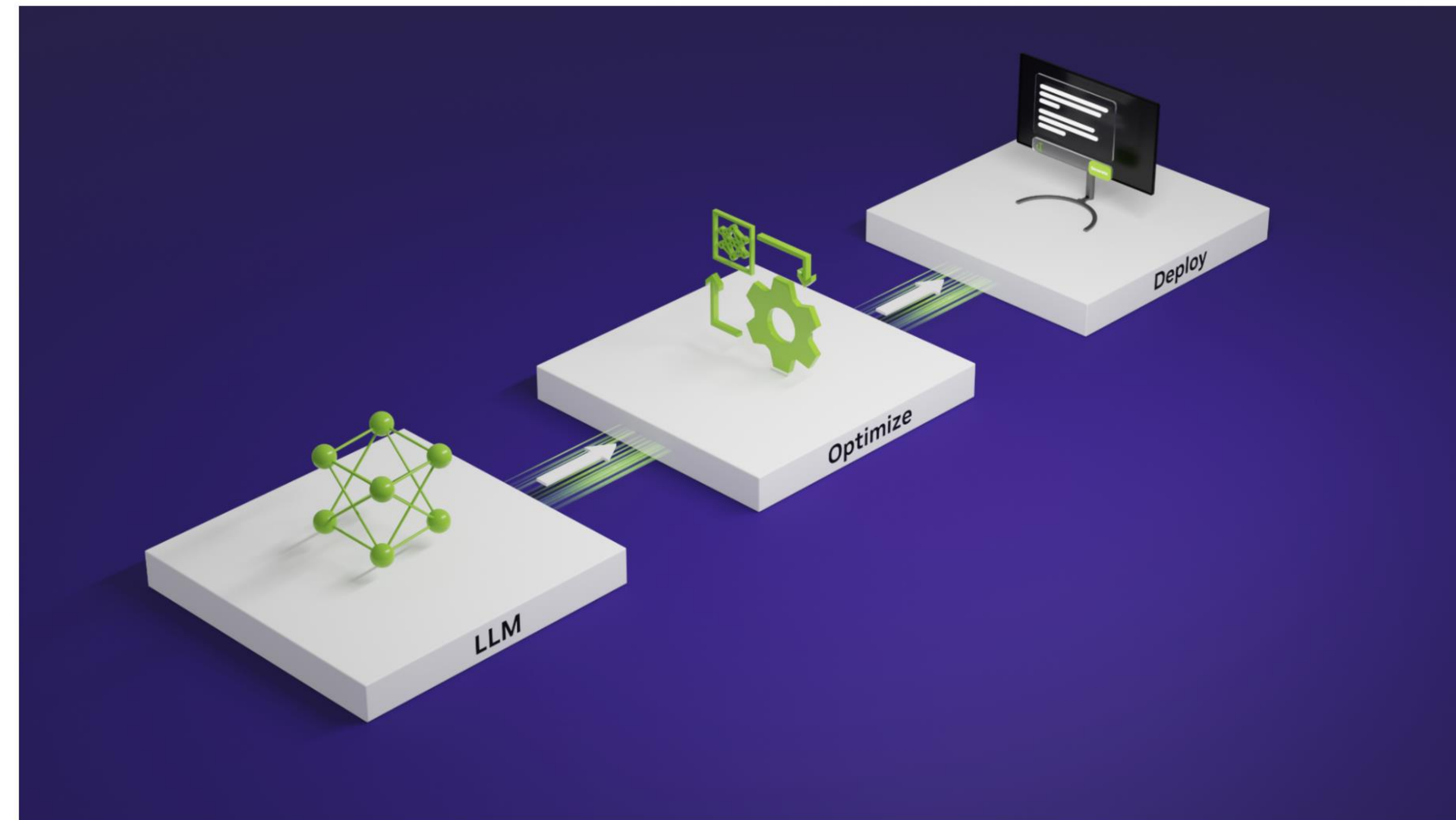
Estimate your KV cache size

Mastering LLM Techniques: Inference Optimization

Nov 17, 2023

+115 Like Discuss (0)

By [Shashank Verma](#) and [Neal Vaidya](#)



Measure

Metrics

Input/Output Sequence Patterns

Advanced Inference Monitoring

Metrics

Use GenAI-Perf For Performance Measurement

Used across all gen-ai project at NVIDIA

GenAI-Perf

GenAI-Perf is a command line tool for measuring the throughput and latency of generative AI models as served through an inference server. For large language models (LLMs), GenAI-Perf provides metrics such as [output token throughput](#), [time to first token](#), [inter token latency](#), and [request throughput](#). For a full list of metrics please see the [Metrics section](#).

Users specify a model name, an inference server URL, the type of inputs to use (synthetic or from dataset), and the type of load to generate (number of concurrent requests, request rate).

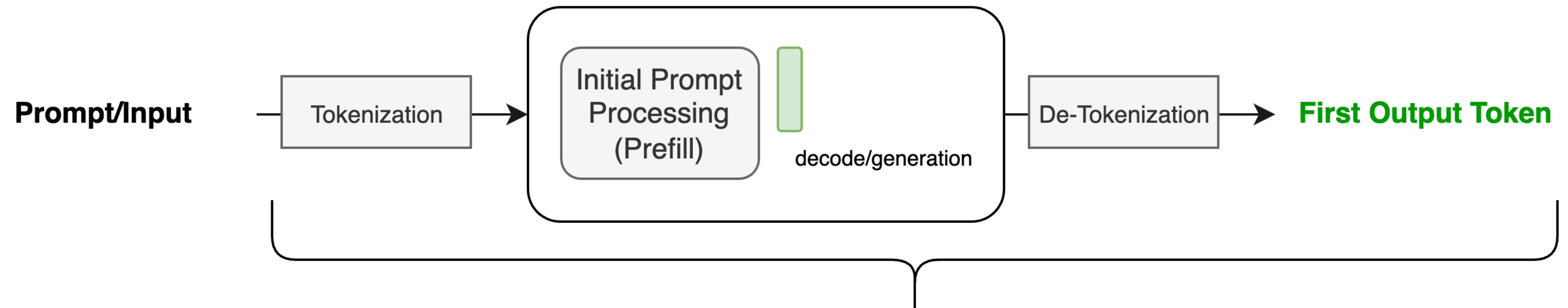
GenAI-Perf generates the specified load, measures the performance of the inference server and reports the metrics in a simple table as console output. The tool also logs all results in a csv file that can be used to derive additional metrics and visualizations. The inference server must already be running when GenAI-Perf is run.

Note

GenAI-Perf is currently in early release and under rapid development. While we will try to remain consistent, command line options and functionality are subject to change as the tool matures.

LLM Inference Measurement – Time to First Token

The time it takes to process the input and generate the first token

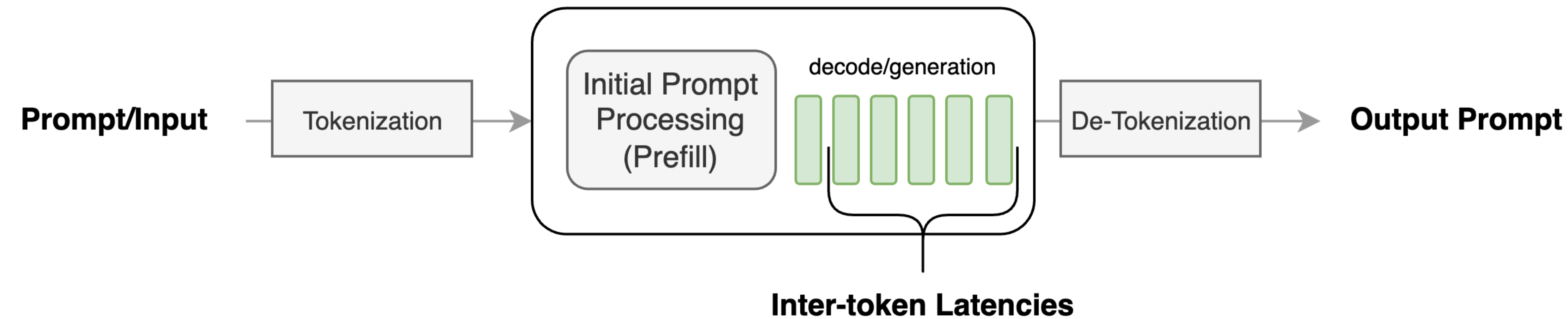


Time to First Token (TTFT)

Signifies time it takes to process the prompt and generate the first token

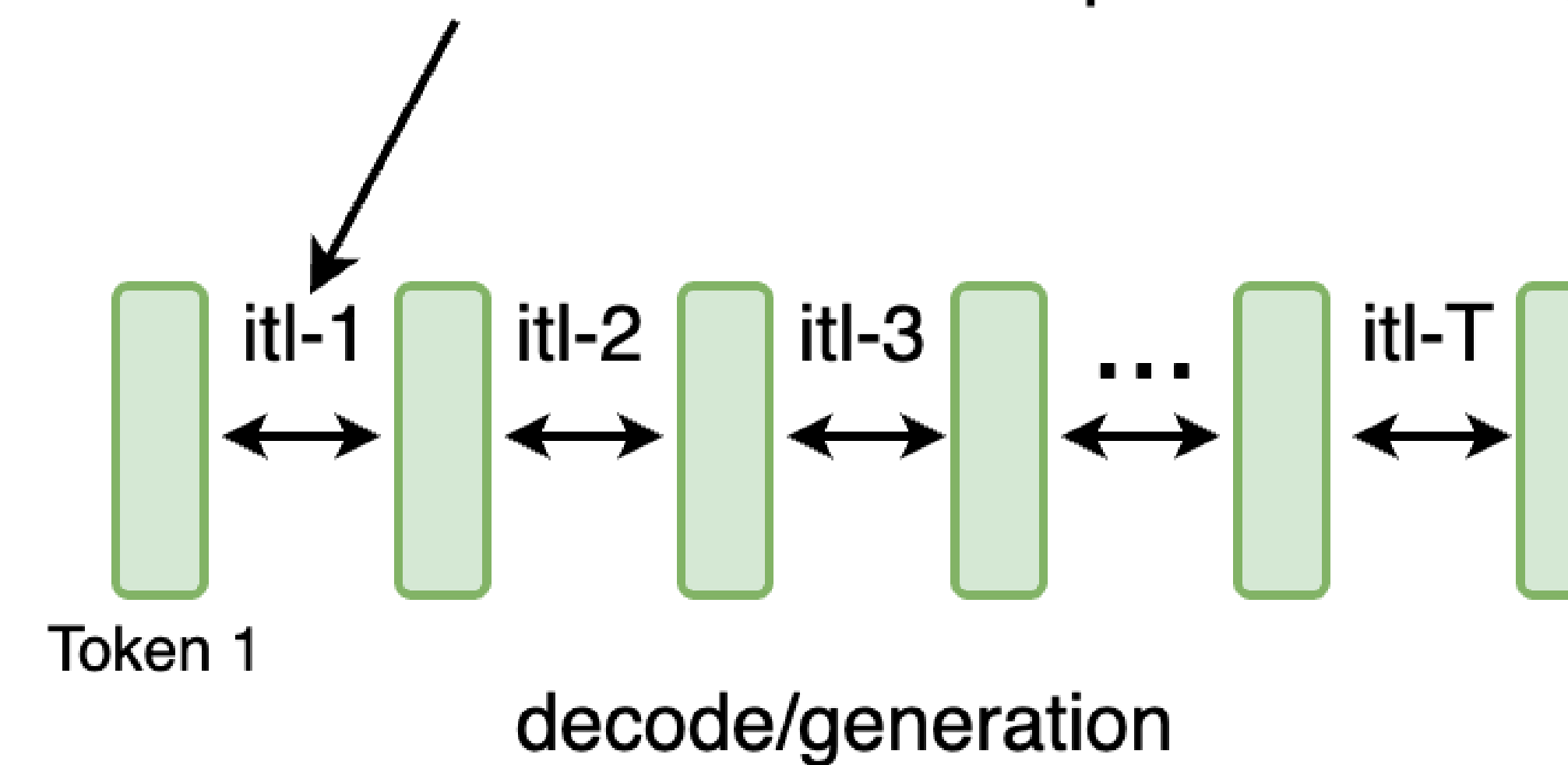
LLM Inference Measurement – Inter-token Latency

Measuring generation performance when the system is under load



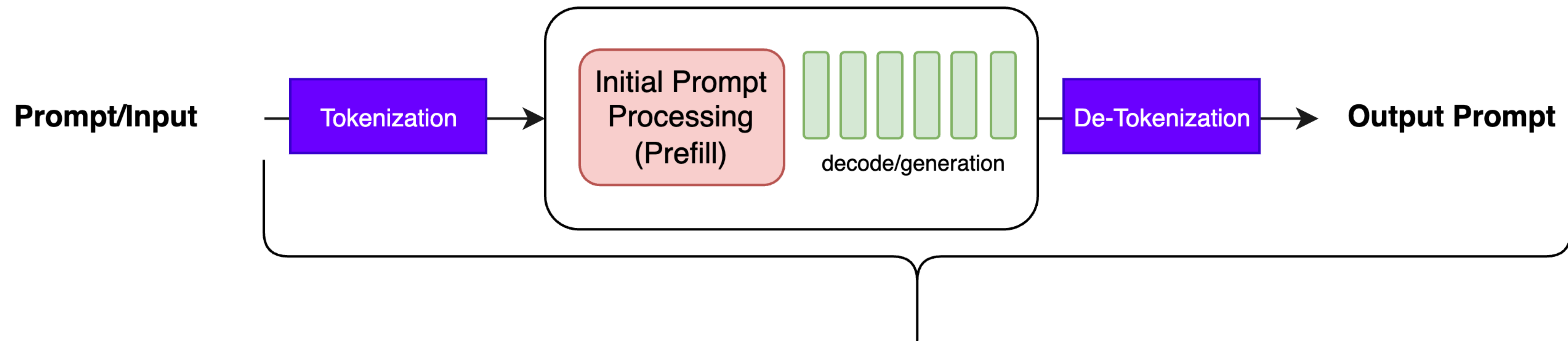
Inter-token Latencies - Should be near constant over all token generations.

Near constant time implies efficient generation



LLM Inference Measurement – Total Time to Generation

The time it takes to process a prompt and generate all the tokens



Total Time to Generation

Measures to total time to process input and generate all tokens

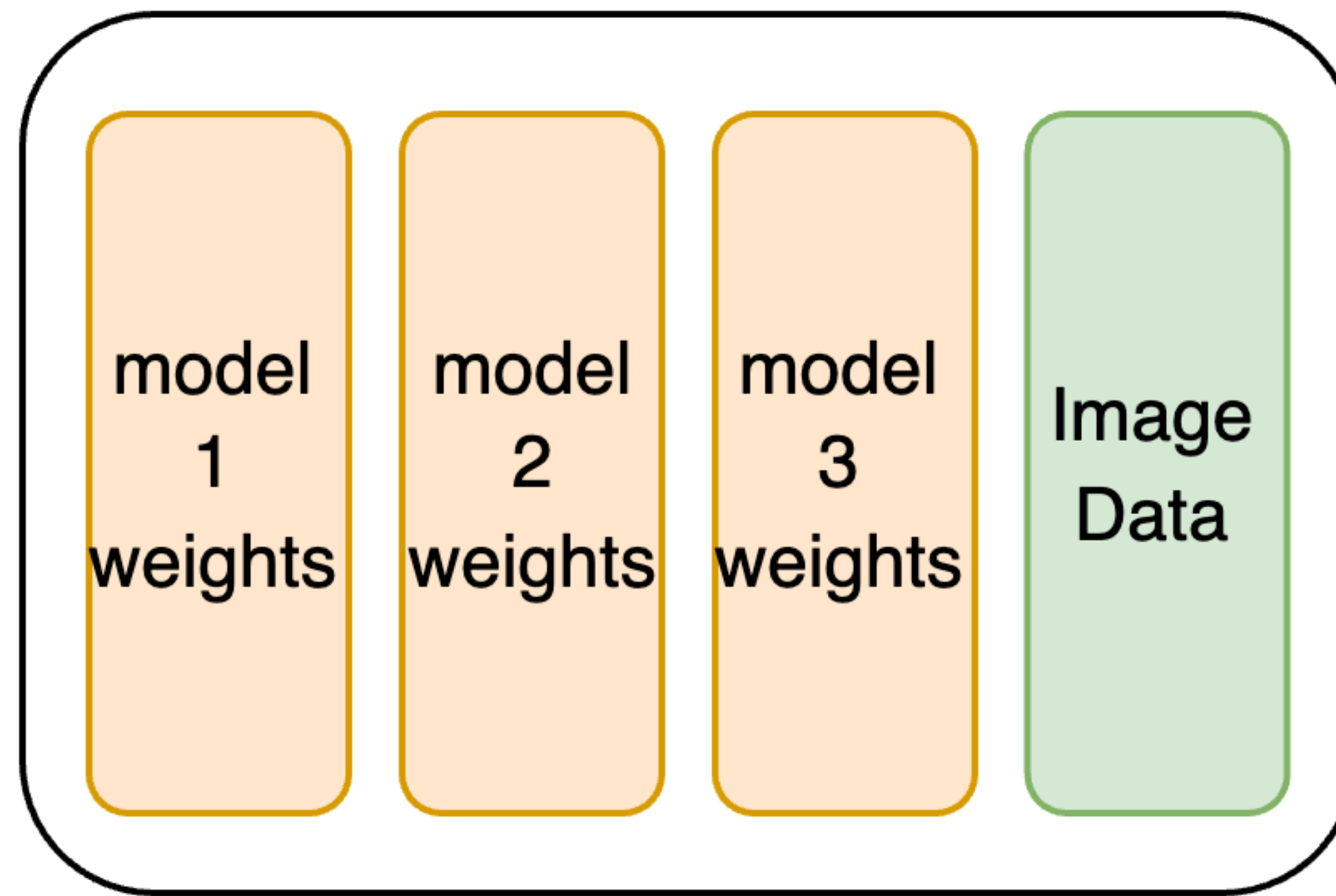
Input/Output Sequence Patterns

ISL - Input Sequence Length

OSL – Out Sequence Length

Why LLM Inference Is Different to Other Inference Scenarios

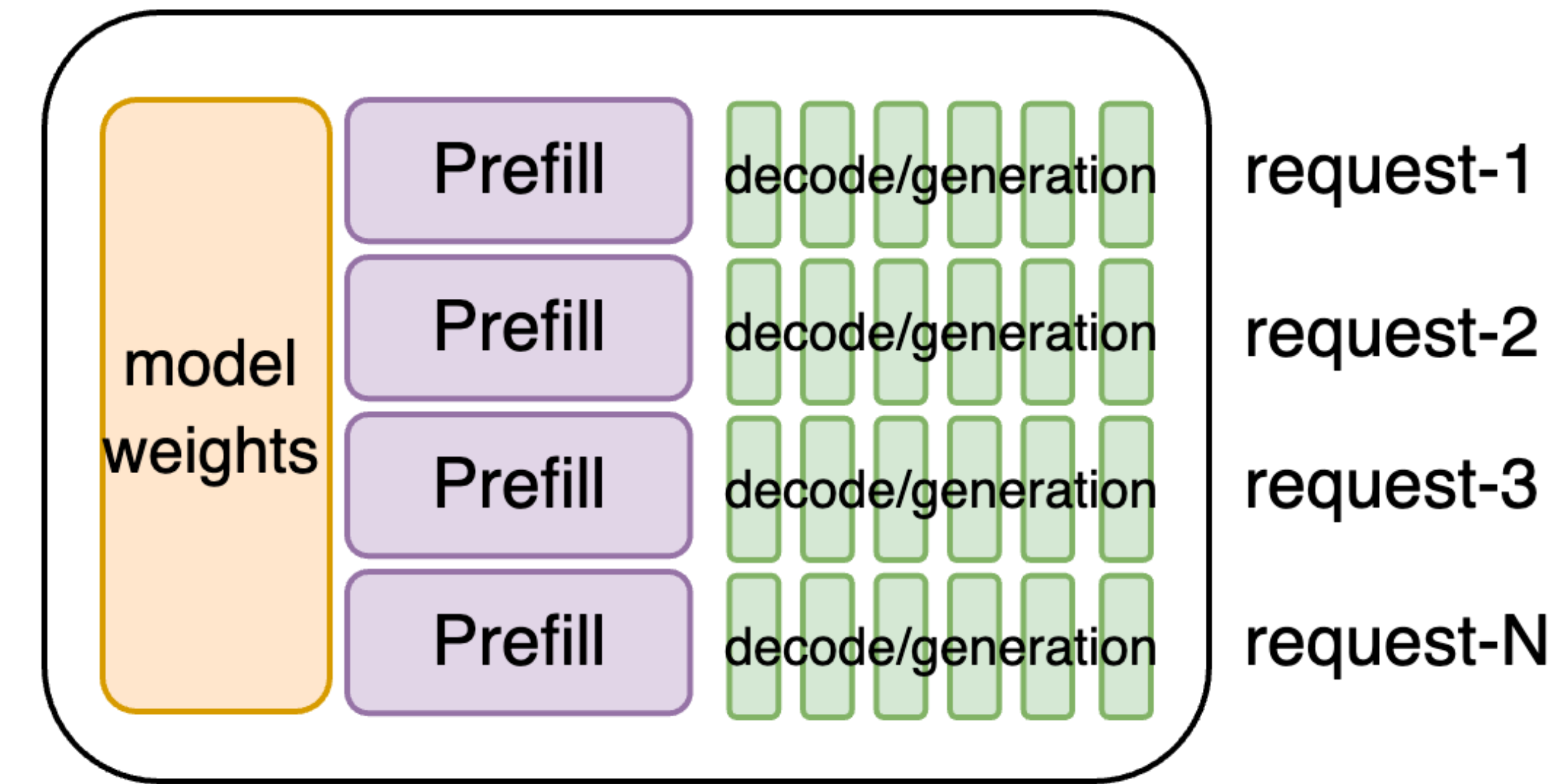
Deep Learning Inference - Computer Vision (for example)



Multiple instances of a model can fit on the single/multiple GPU(s).

More model copies per GPU in regular deep learning inference increases throughput significantly.

LLM Inference



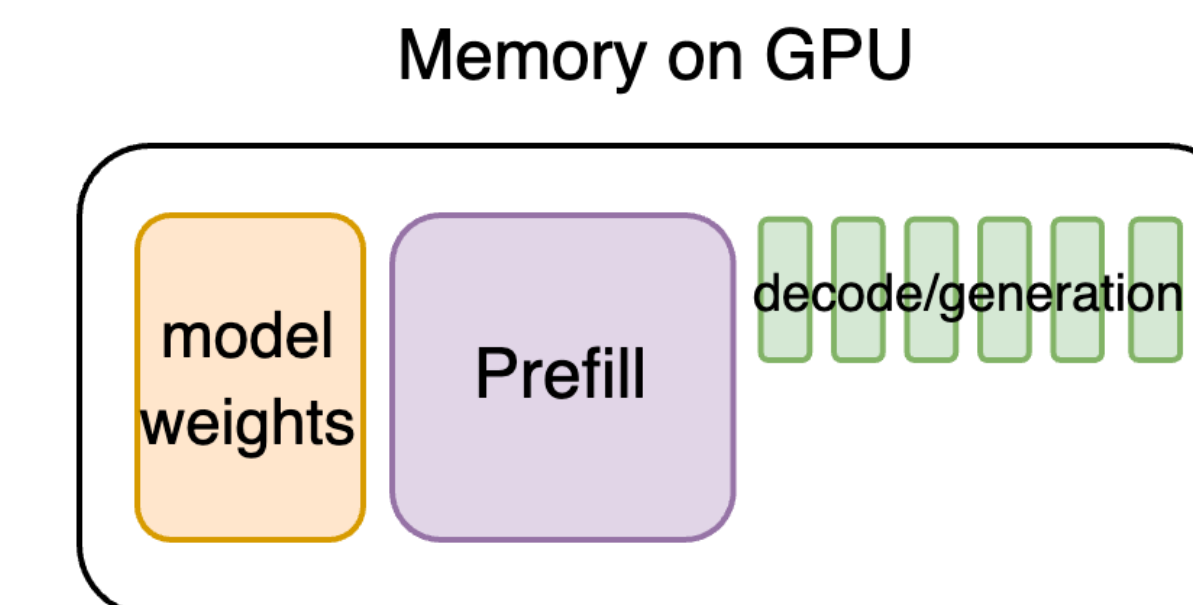
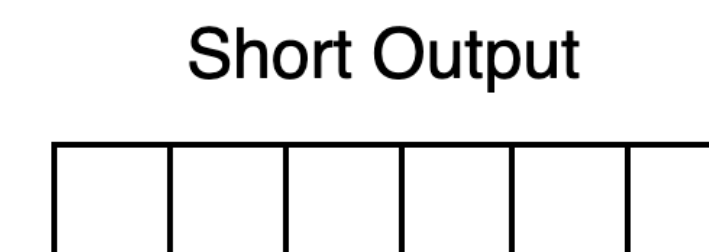
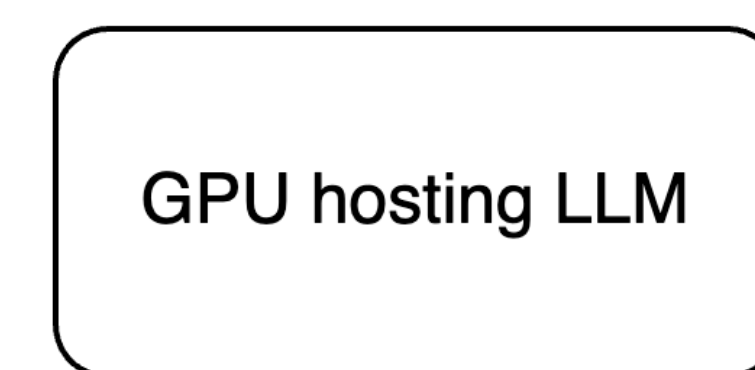
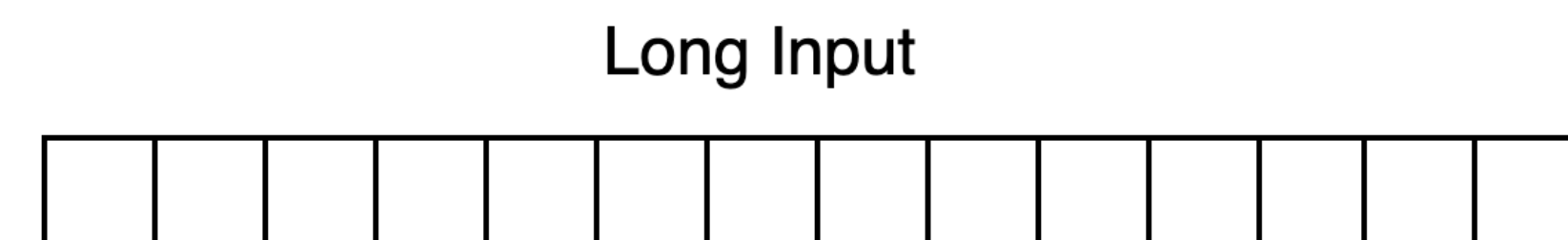
Single instance of the LLM model per GPU(s)

For LLM Inference the memory remaining after model weights is reserved for tokens across all requests.

How Prompt Size Affects Memory Usage

Memory on the GPU is filled up with tokens from prefill and generation

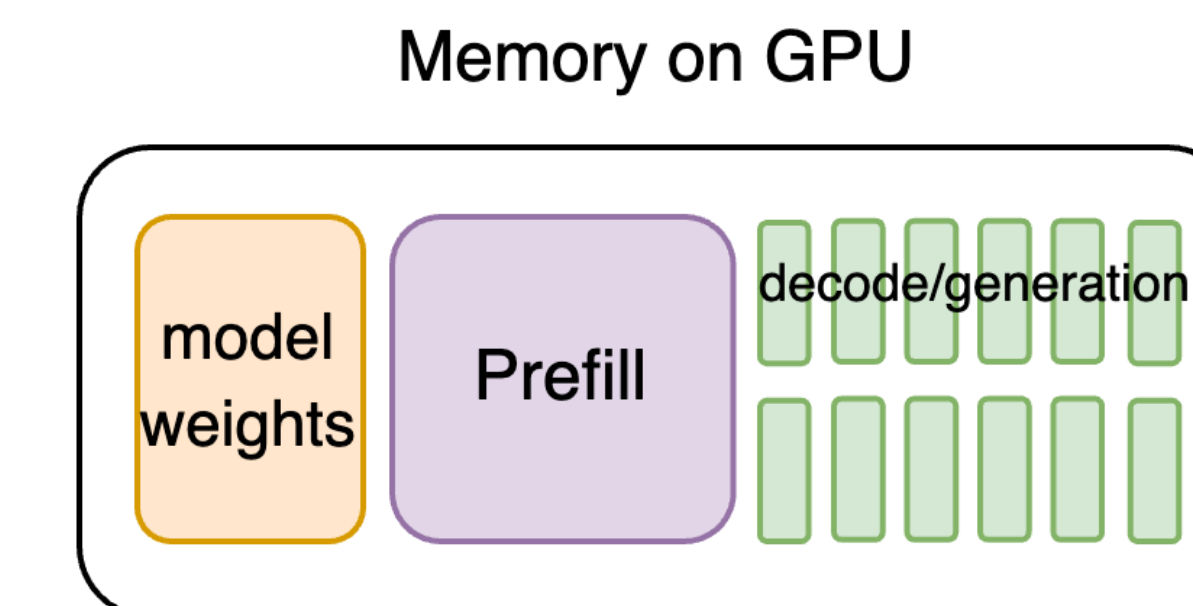
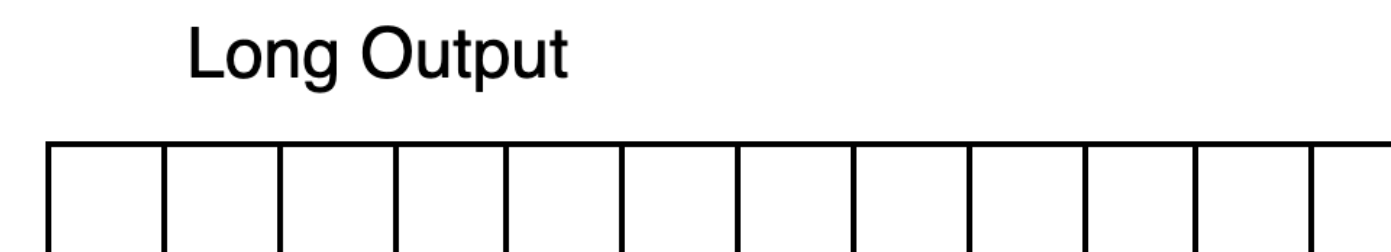
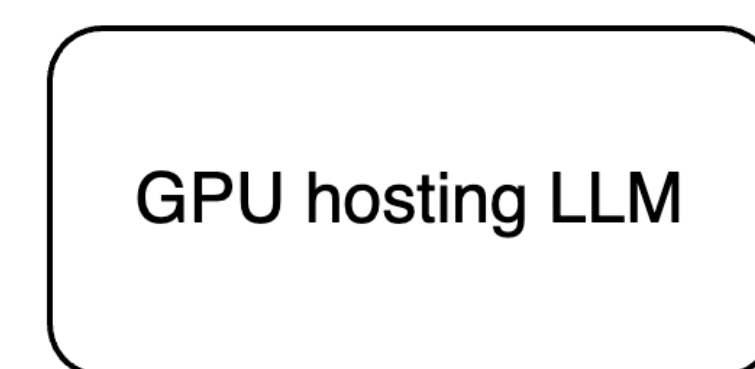
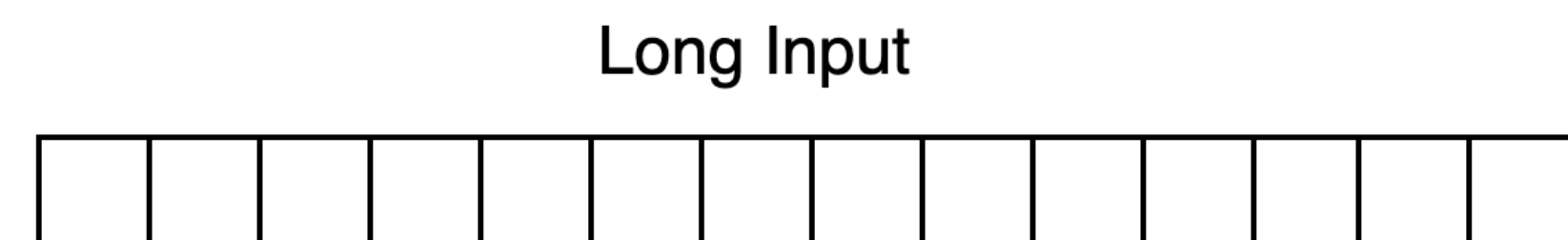
LISO - Long Input Short Output



Use Cases

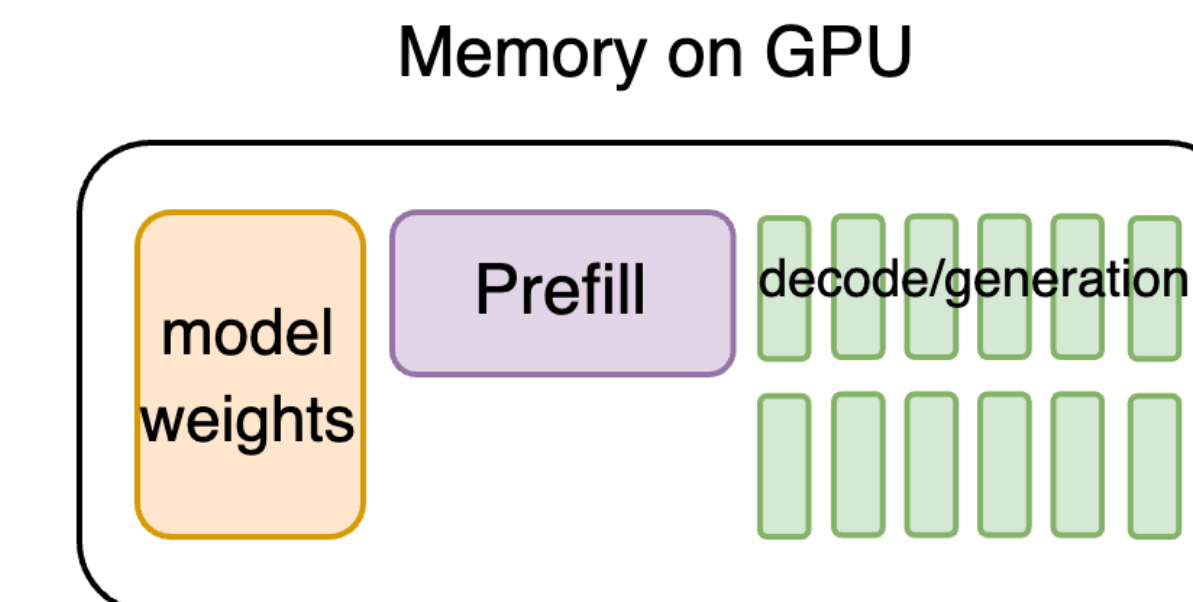
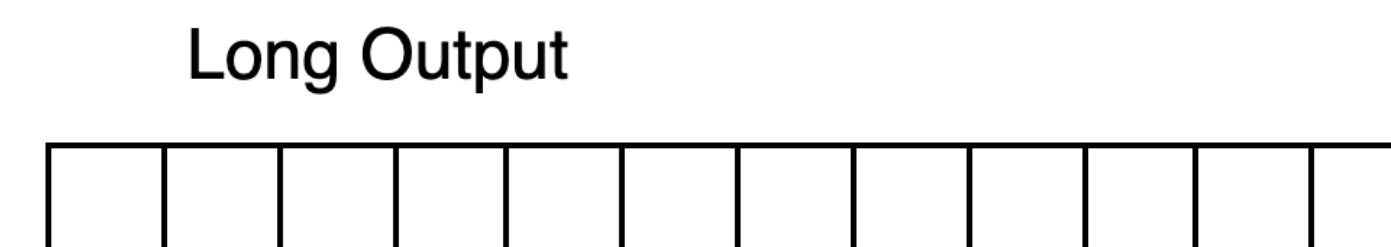
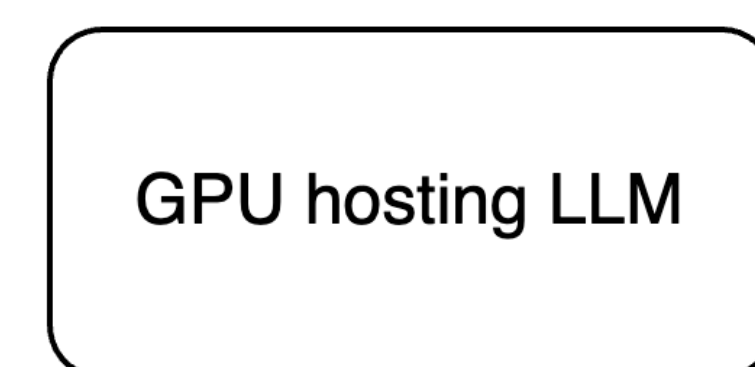
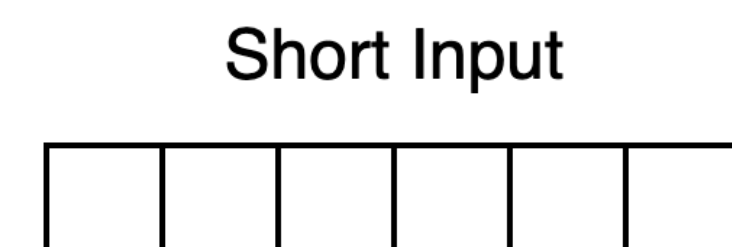
Summarization
Meeting Recap

LILO - Long Input Long Output



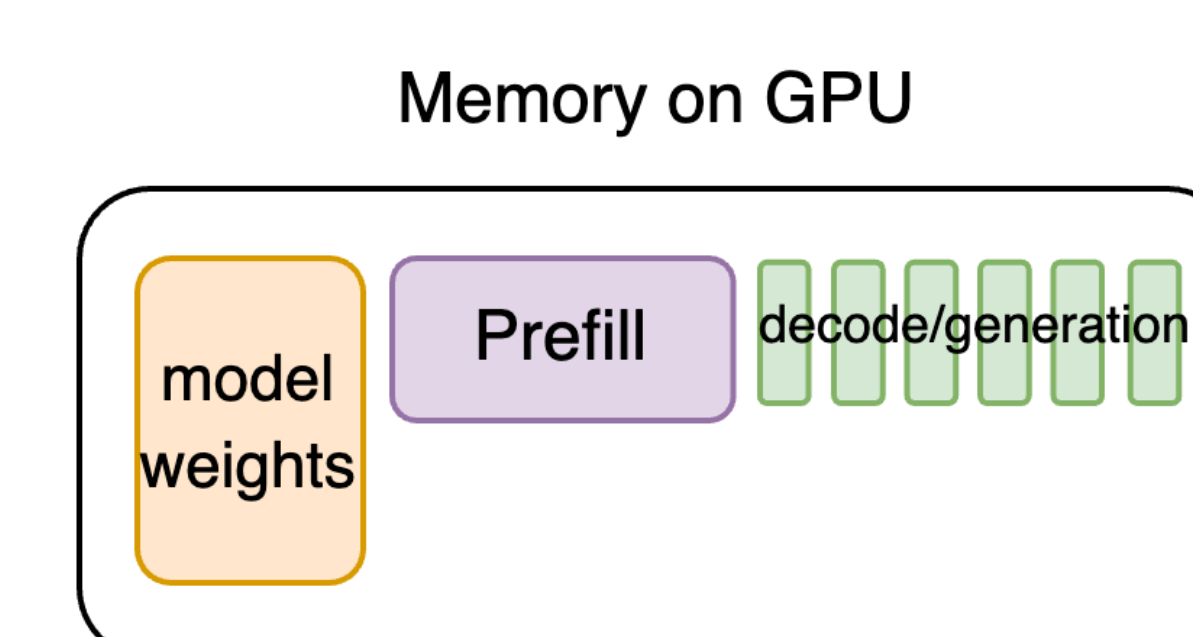
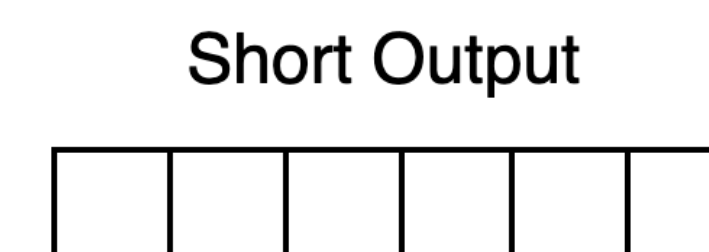
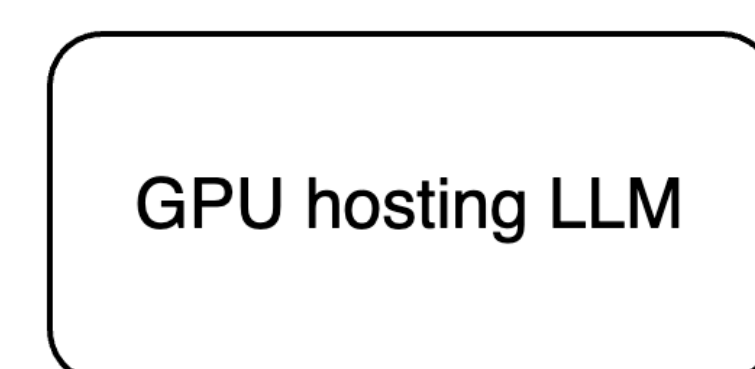
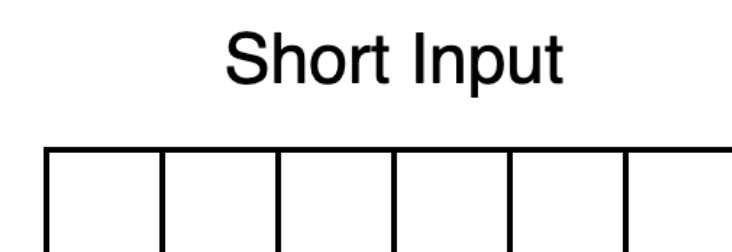
Rewrite/edit documents

SILO - Short Input Long Output



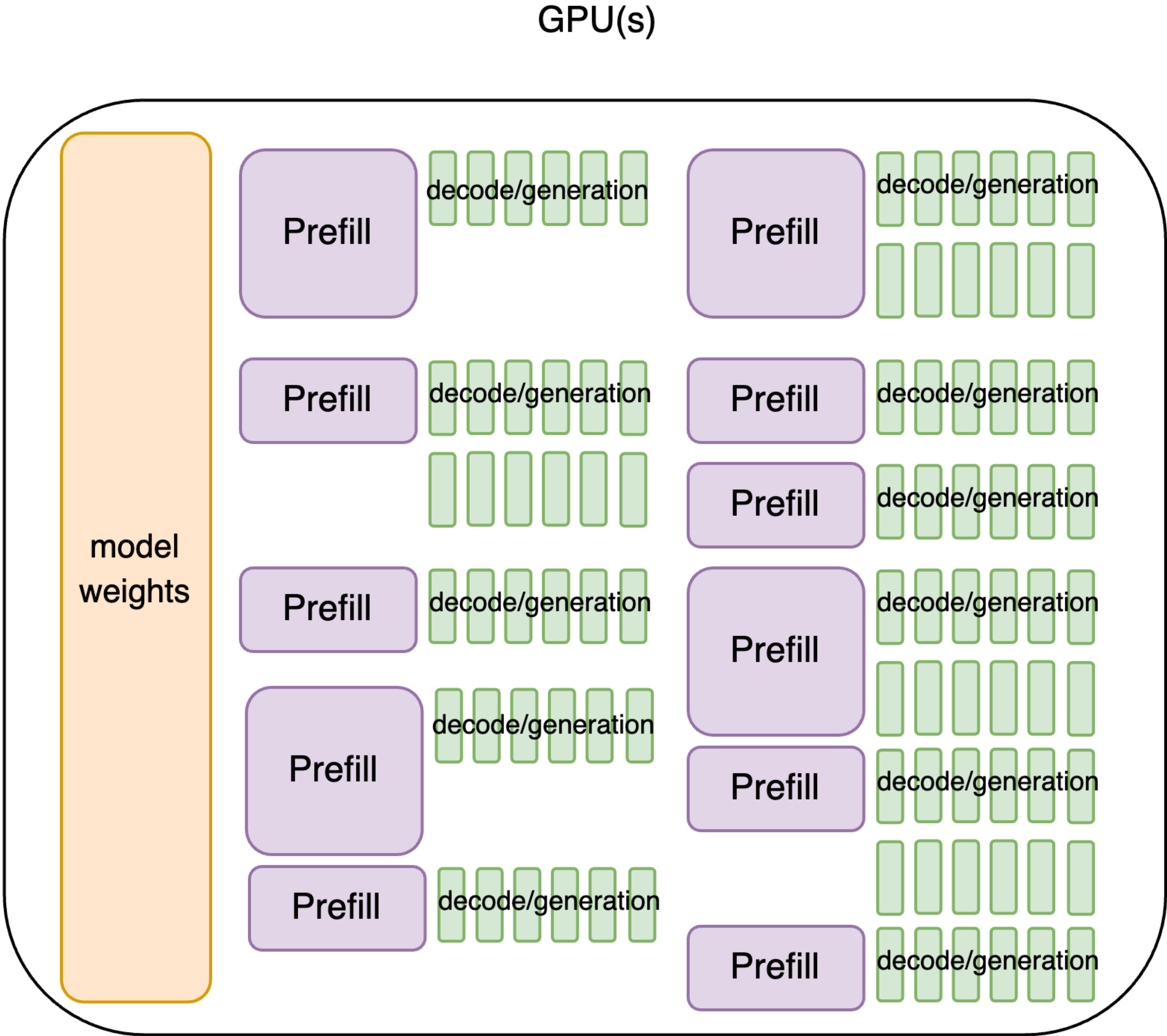
Writing an essay

SISO - Short Input Short Output



short q/a

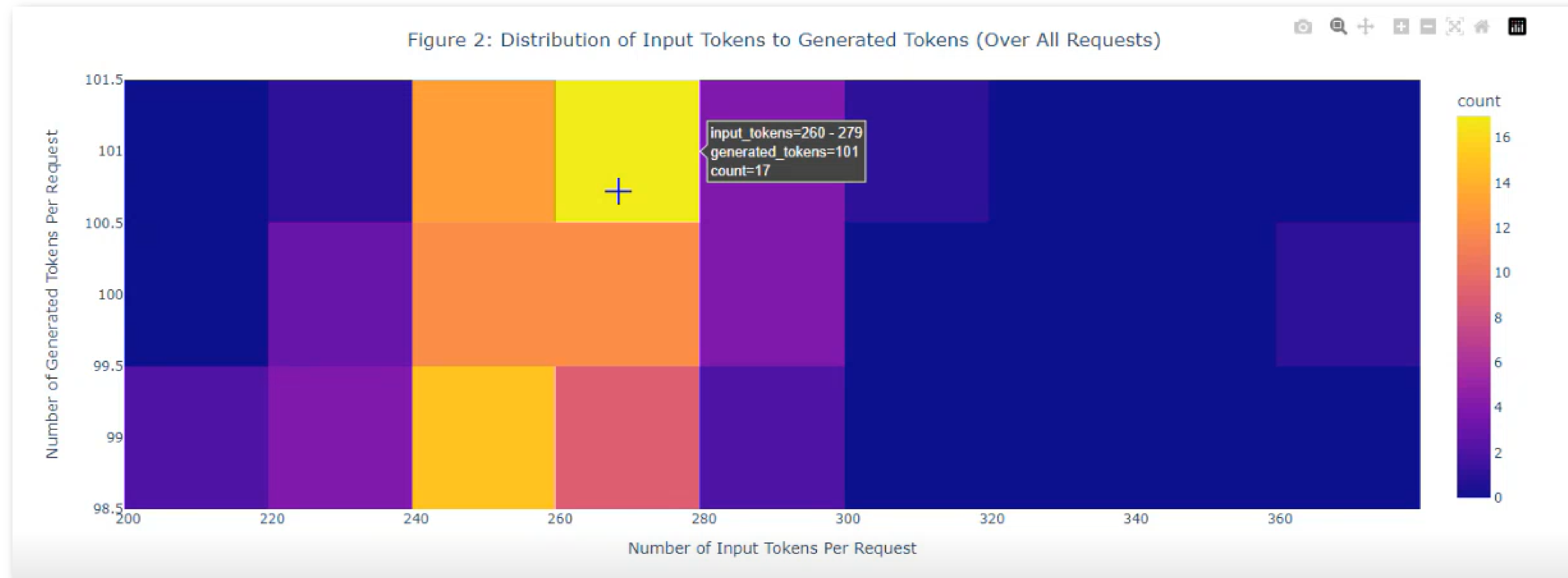
Distribution Of User Prompts and Generations Occupying GPU Memory



GPU hosting an LLM filled with requests that have different Input Sequence Lengths (ISL) and Output Sequence Lengths (OSL)

Advanced Inference Monitoring

Understanding Inference Request Profiles



2D histogram of Input Sequence Length vs Output Sequence Length across all requests

Data is used to build more optimized engines with statistical certainty

Time to First Token Analysis



Box plot of Time-to-First Token Values across all requests

Measures the performance of the prefill stage and KV Cache efficiency

Time to Completion Analysis



Box plot of Total-Generation Time across all requests

Used to measure overall performance of deployment and generation phases

Token-to-Token Latency vs Token Position

Measuring the performance of generation



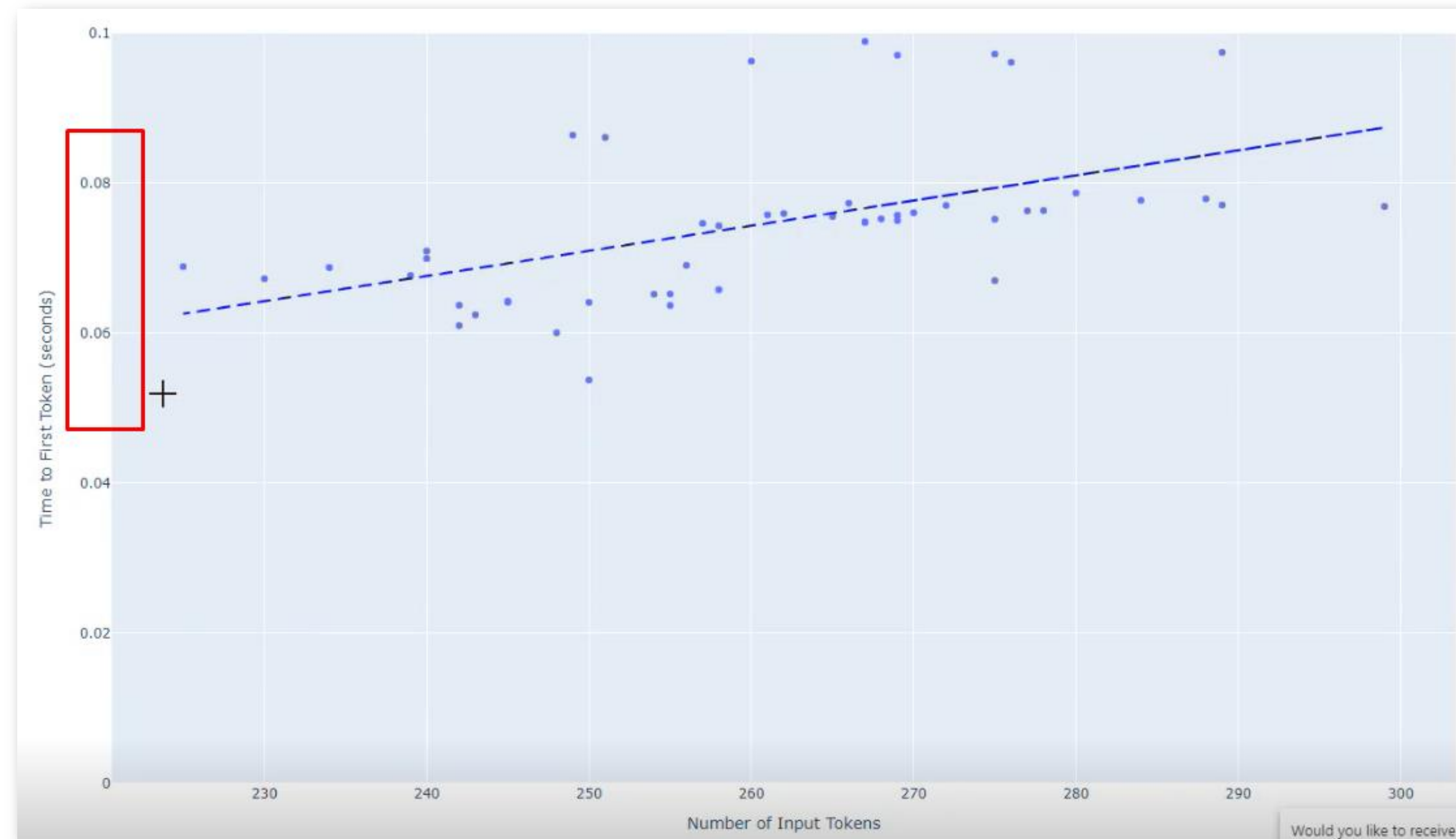
Plot of Token Position vs Inter-token Latency

As the OSL grows you want to see minimal variation in Inter-token Latencies

Great visual of performance under load

Time to First Token vs Number of Input Tokens

Understanding and modeling prefill performance



Plot of Input Sequence Length vs Time-to-First Token across all requests

Measures the performance of the prefill stage as the ISL grows

Smaller slope indicates more efficient prefill

Optimize

TRT-LLM + Triton

NIM – NVIDIA Inference Microservice

TensorRT-LLM Optimizing LLM Inference

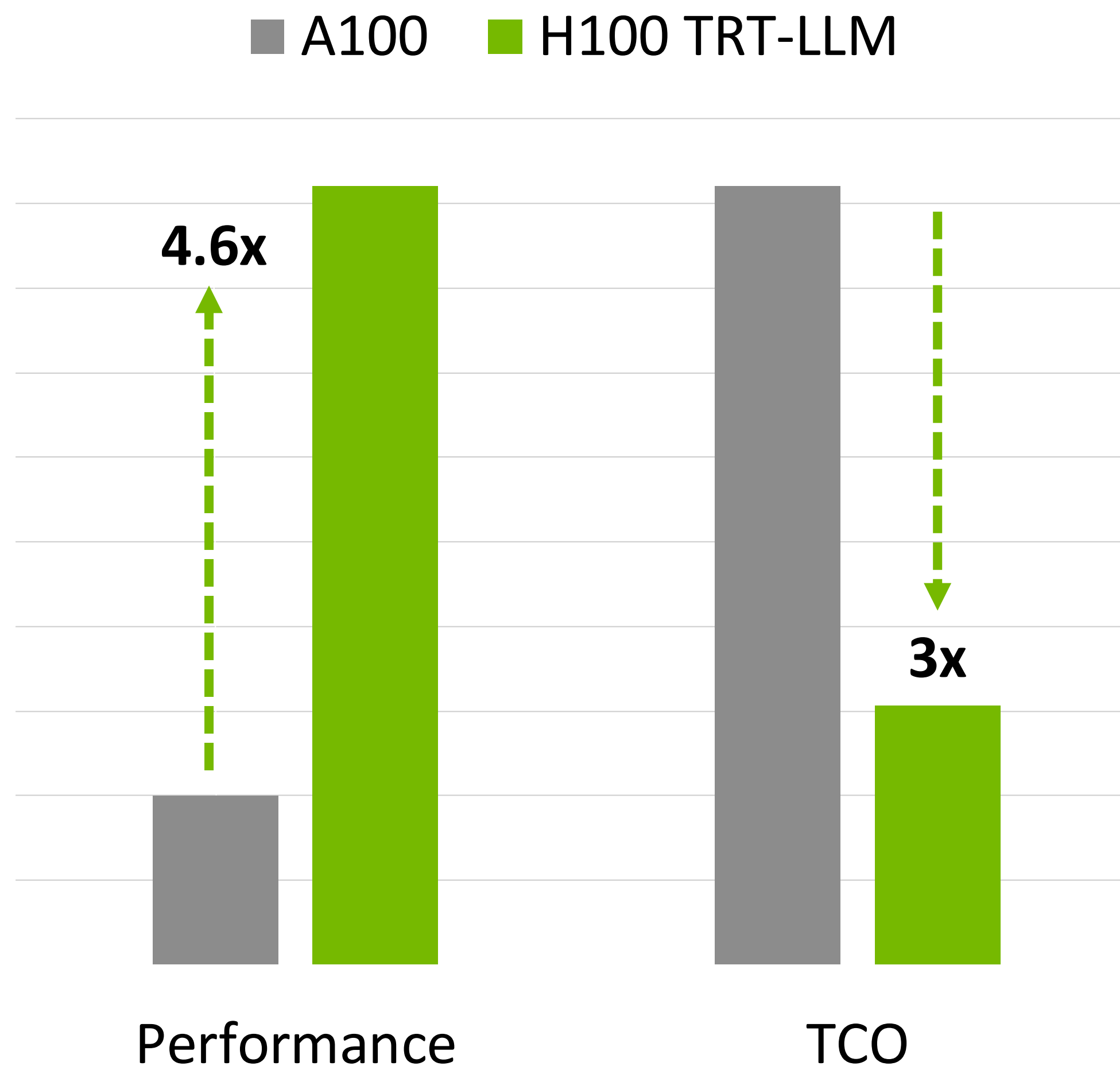
SoTA Performance for Large Language Models for Production Deployments

TensorRT-LLM is an **open-source** library to **optimize inference performance** on the latest **Large Language Models** for NVIDIA GPUs.

- Built on FasterTransformer and TensorRT
- Simple Python API for defining, optimizing, & executing LLMs for inference in production.

SoTA Performance

Leverage TensorRT compilation & kernels from FasterTransformers, CUTLASS, OAI Triton, ++



Ease Extension

Add new operators or models in Python to quickly support new LLMs with optimized performance

```
# define a new activation
def silu(input: Tensor) → Tensor:
    return input * sigmoid(input)

#implement models like in DL Fws
class LlamaModel(Module)
    def __init__(...)
        self.layers = ModuleList([...])

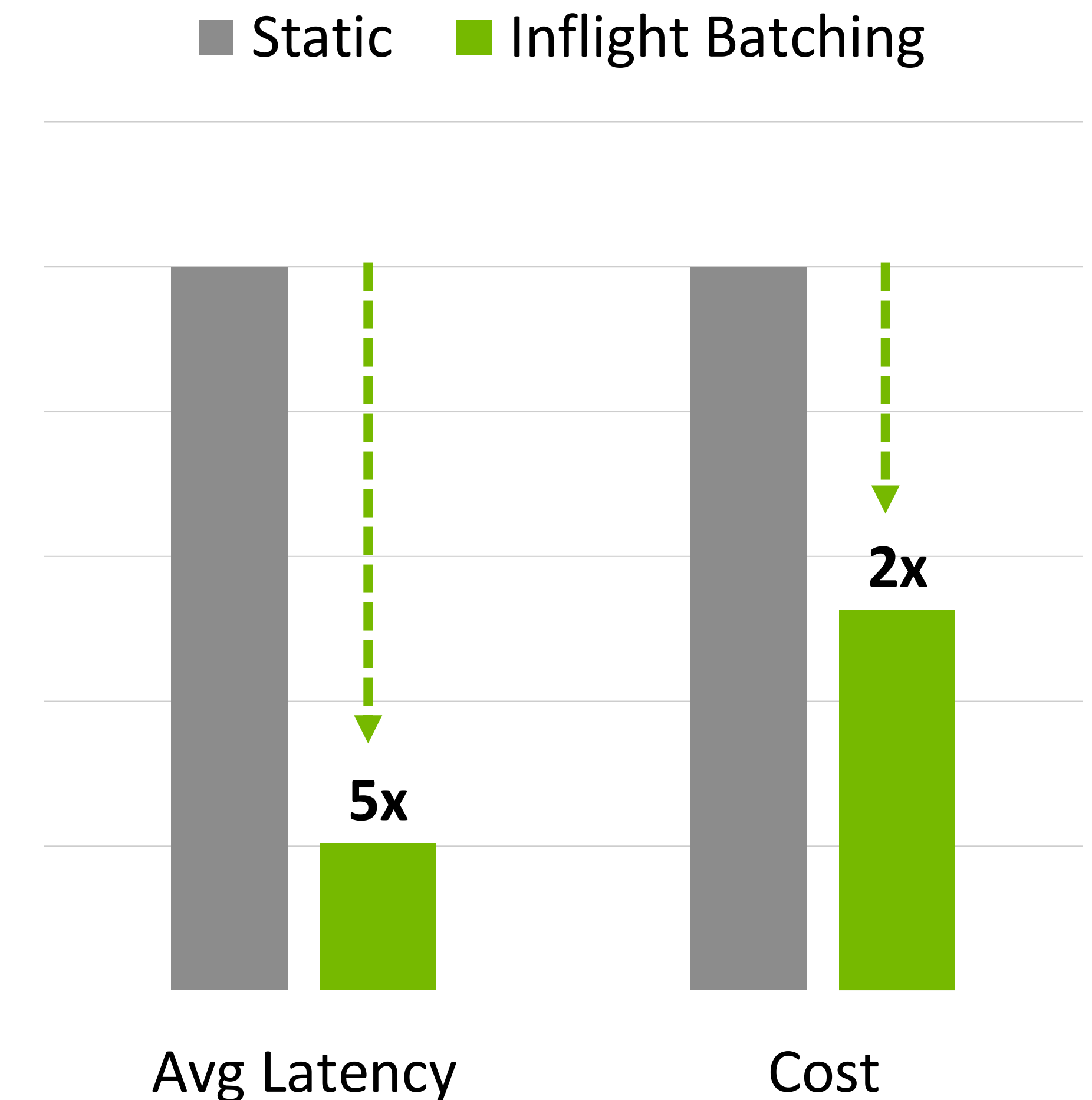
    def forward (...)
        hidden = self.embedding(...)

        for layer in self.layers:
            hidden_states = layer(hidden)

        return hidden
```

LLM Batching with Triton

Maximize throughput and GPU utilization through new scheduling techniques for LLMs



TRT-LLM Features

KV Cache and Attention Optimizations

Paged KV Cache

Quantized KV Cache
INT8/FP8

Multi-head Attention

Multi-Query Attention

Group Query
Attention

Sliding Window
Attention

Streaming LLMs

KV Cache Reuse

System Prompt
Cache

Lora and Customizations

Multiple Loras

Prompt Tuned Models

SLoRA/LoRAx

Scaling and Distributed

Multi-GPU

Multi-Node

Tensor and Pipeline
Parallelism

Quantization

Smooth Quant

FP8

AWQ - Activation Aware
Weight Quantization

GPT-Q

Inflight Batching

Multi-modal Support

TRT-LLM Models

[See online repo for more models](#)

artic

baichuan

bert

blip2

bloom

chatglm

cogvlm

dbRX

DiT

encoder-decoder

falcon

gemma

gpt

gptj

gptneox

grok

InternLM

jais

llama - All

mamba

medusa

mixtral

nemotron

smaug

whisper

TensorRT-LLM Execution

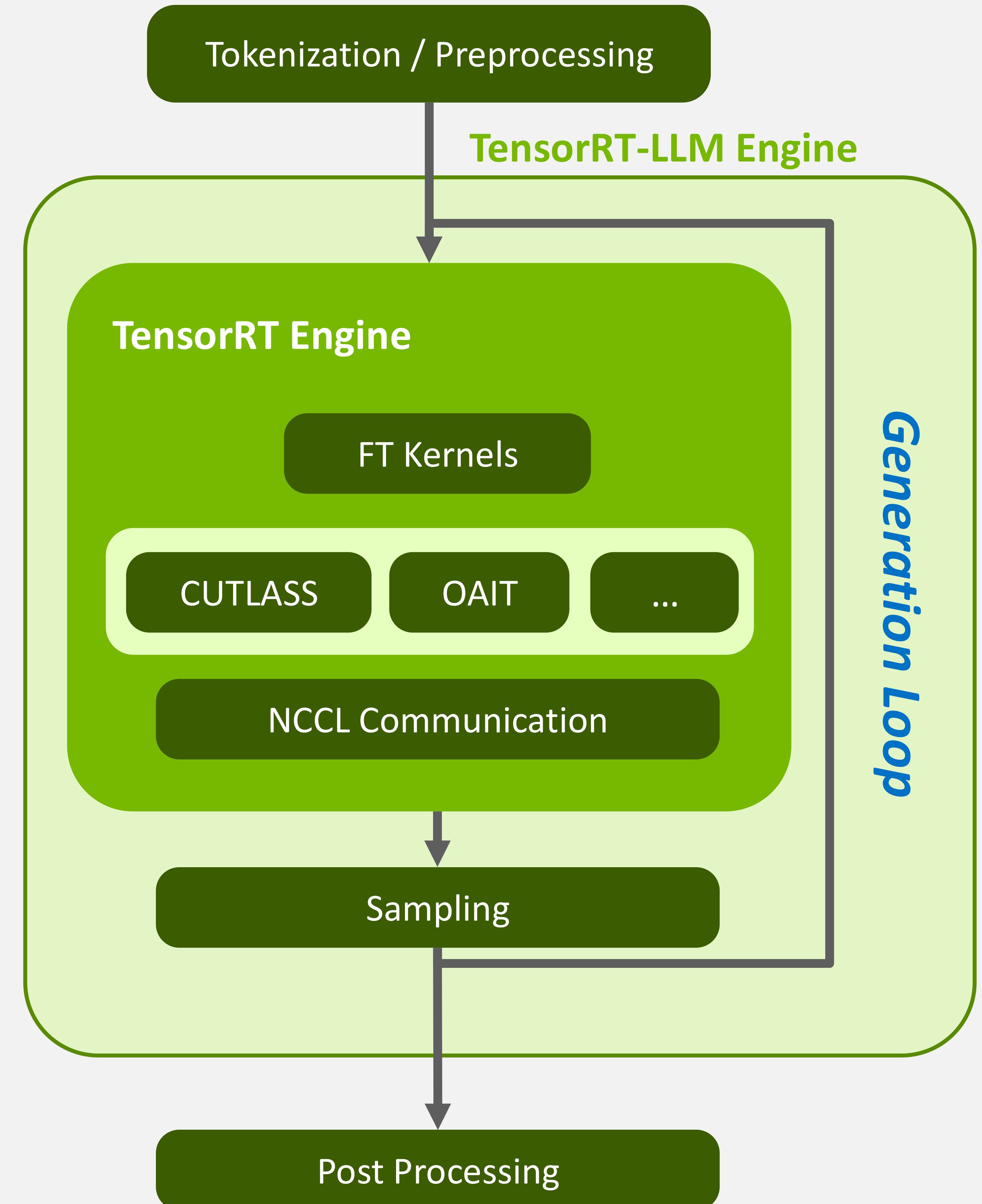
Built on top of FT & TensorRT

TensorRT Engine accelerates the Transformer Architecture

- Executes both TensorRT compiled graph & Plugins
- Handles all plugin management & GPU communication

TensorRT-LLM handles the entire LLM

- Sampling runs on GPU for best performance loop
- Handles auto-regressive loop & batching
- Tokenization / detokenization can all be used directly



Optimized Models With TRT-LLM

TRT-LLM builds an inference engine specific to a GPU

An A100 Inference Engine won't work on H100

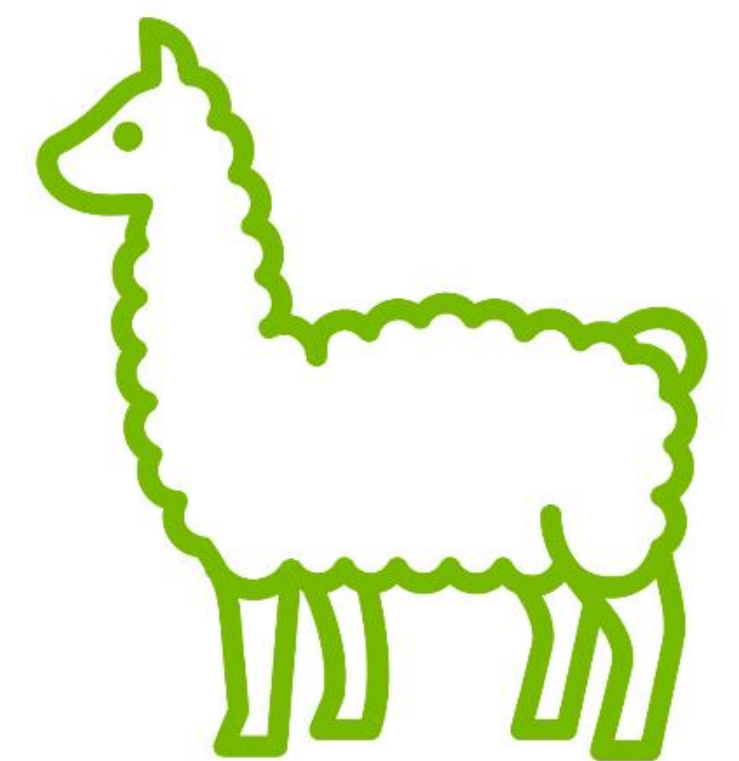
Hopper and Ada Lovelace GPUs have FP8 -> smaller, faster engines



Base Llama model before optimization



NVIDIA LLama Inference Engine on A100



NVIDIA LLama Inference Engine on H100

TensorRT-LLM & Triton

Deploying a model with Triton Server

Allows serving models through Triton Inference server

- All key features (IFB, PagedKV, Speculative Decoding, ++)
- Available on Github & NGC Containers

0. Prepare Pre-reqs

TRT-LLM Engine

Triton TRT-LLM Backend

1. Create Model Repository

2. Launch Server

3. Launch Client

4. Request!

Where Inference Performance Is Headed

FP16



FP8

Ada Lovelace
Hopper



FP4

Blackwell

Reduces memory, increases speed, improves TCO

TensorRT-LLM GPU Support

Hopper to Volta



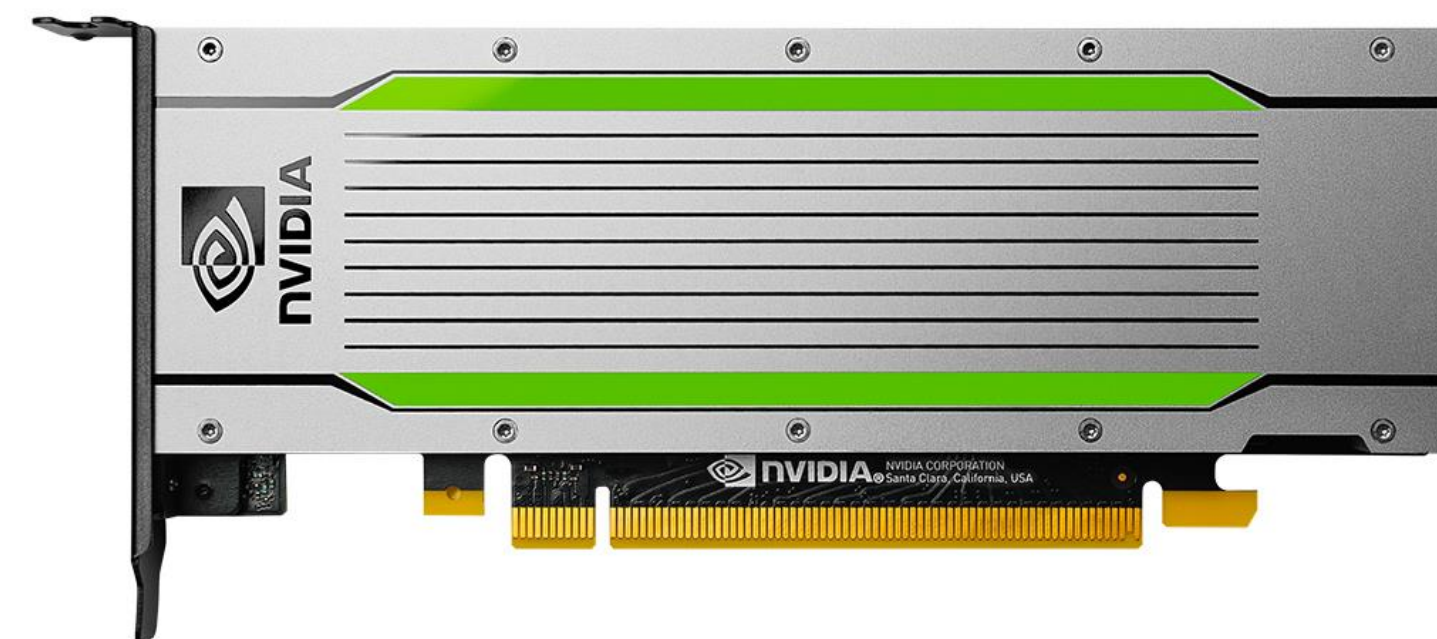
Hopper
GH200, H200, H100, H20



Ada
L4, L40, L40S, L20



Ampere
A100, A30, A10



Turing
T4 (Experimental)



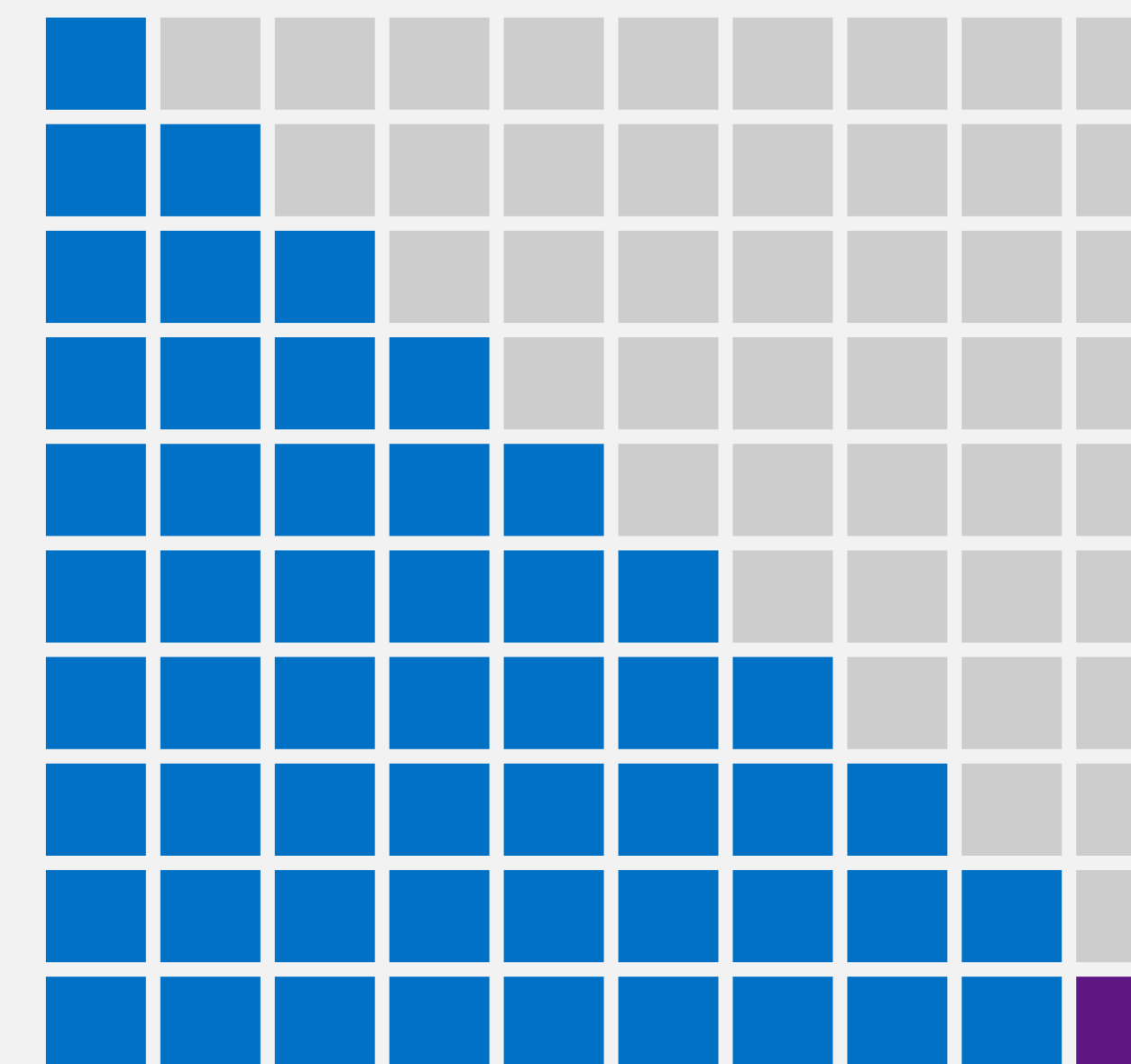
Volta
V100 (Experimental)

KV Cache & Attention Techniques

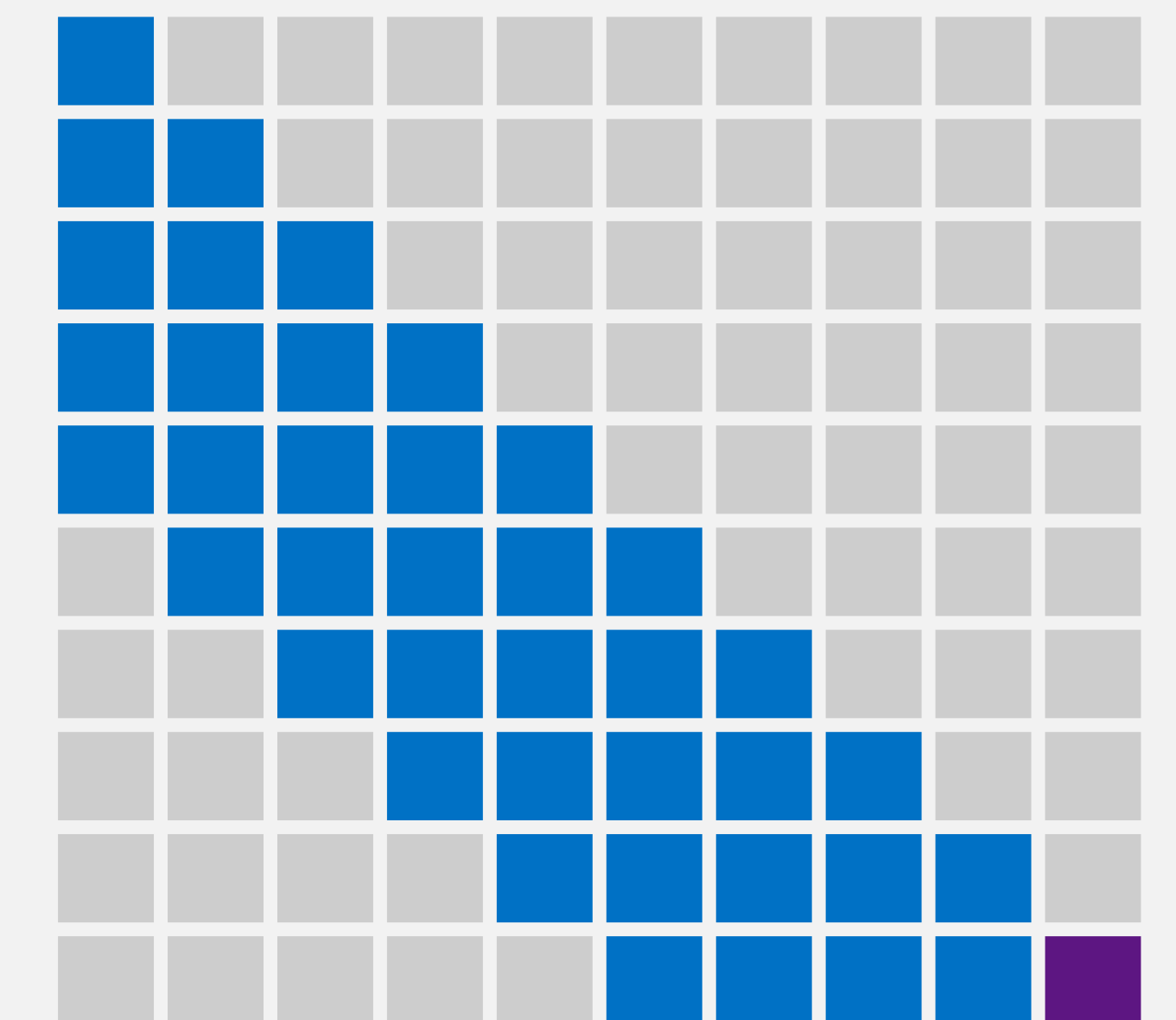
(Sliding) Window Attention, & Streaming LLM

- Allow for longer (sometimes unlimited) sequence length
 - Reduces KV Cache Memory usage
 - Avoids OOM Errors
- (Sliding) Windowed Attention evict tokens based on arrival
 - Significantly reduces memory usage
 - Can negatively impact accuracy or require recomputing KV
- Streaming-LLM allows for unlimited sequence length
 - Does not evict Attention Sinks (important elements)
 - KV Cache stays constant size
 - Does not require recompute & does not impact accuracy
 - Particularly beneficial for multi-turn (ie. chat) usecases

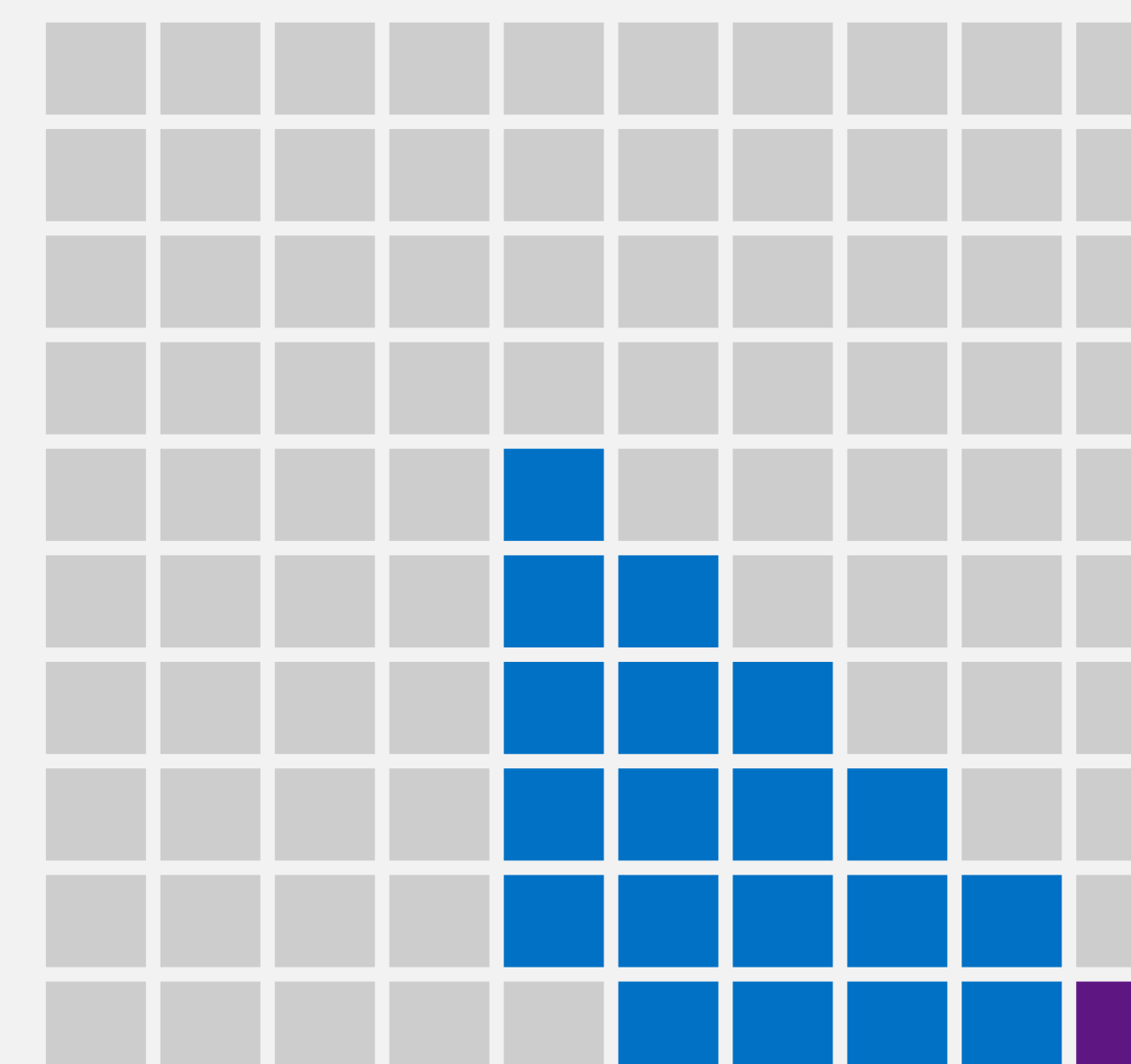
Attention KV Cache Usage (*Less is Better*)



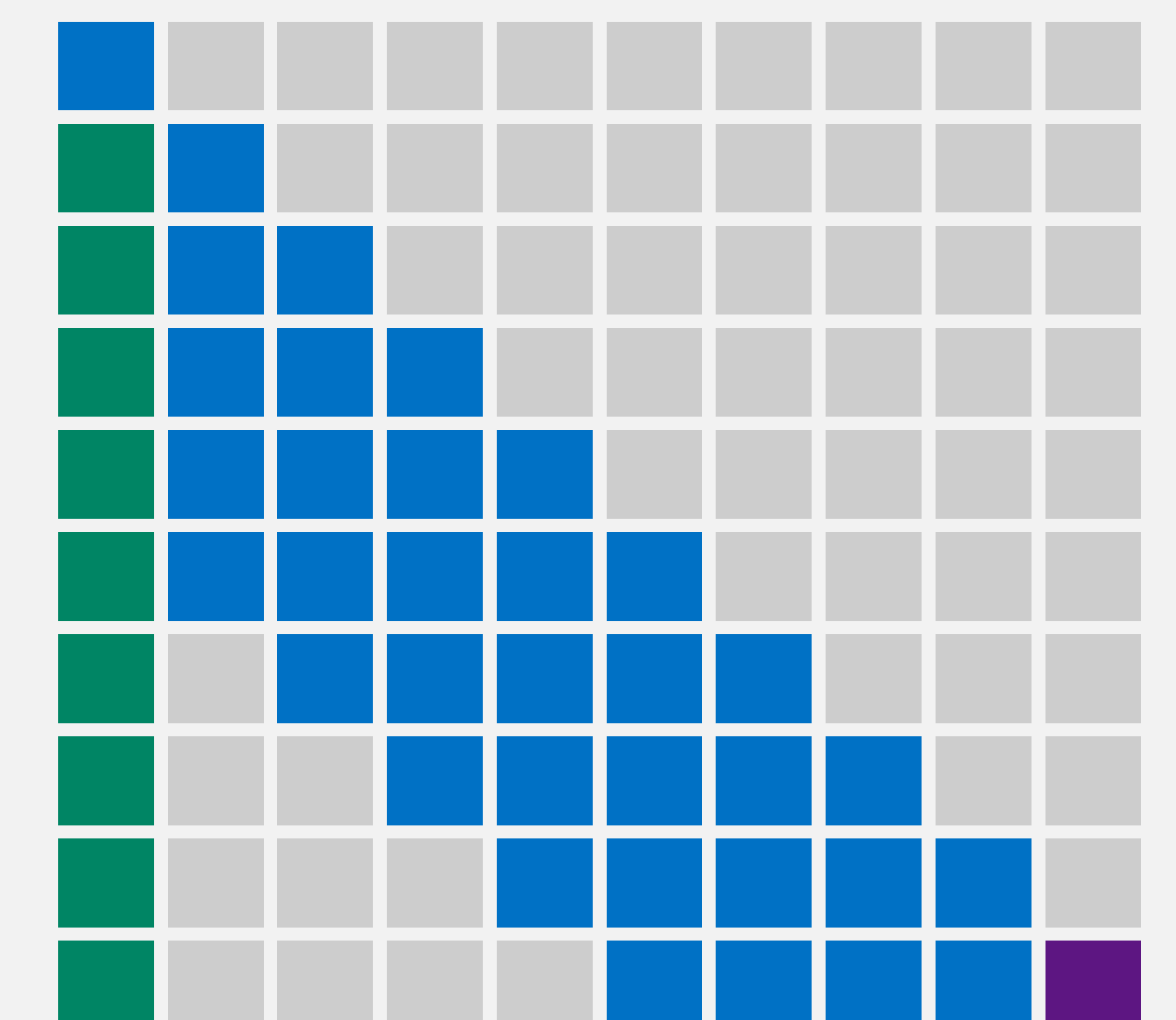
Dense



Windowed



Sliding Window



StreamingLLM

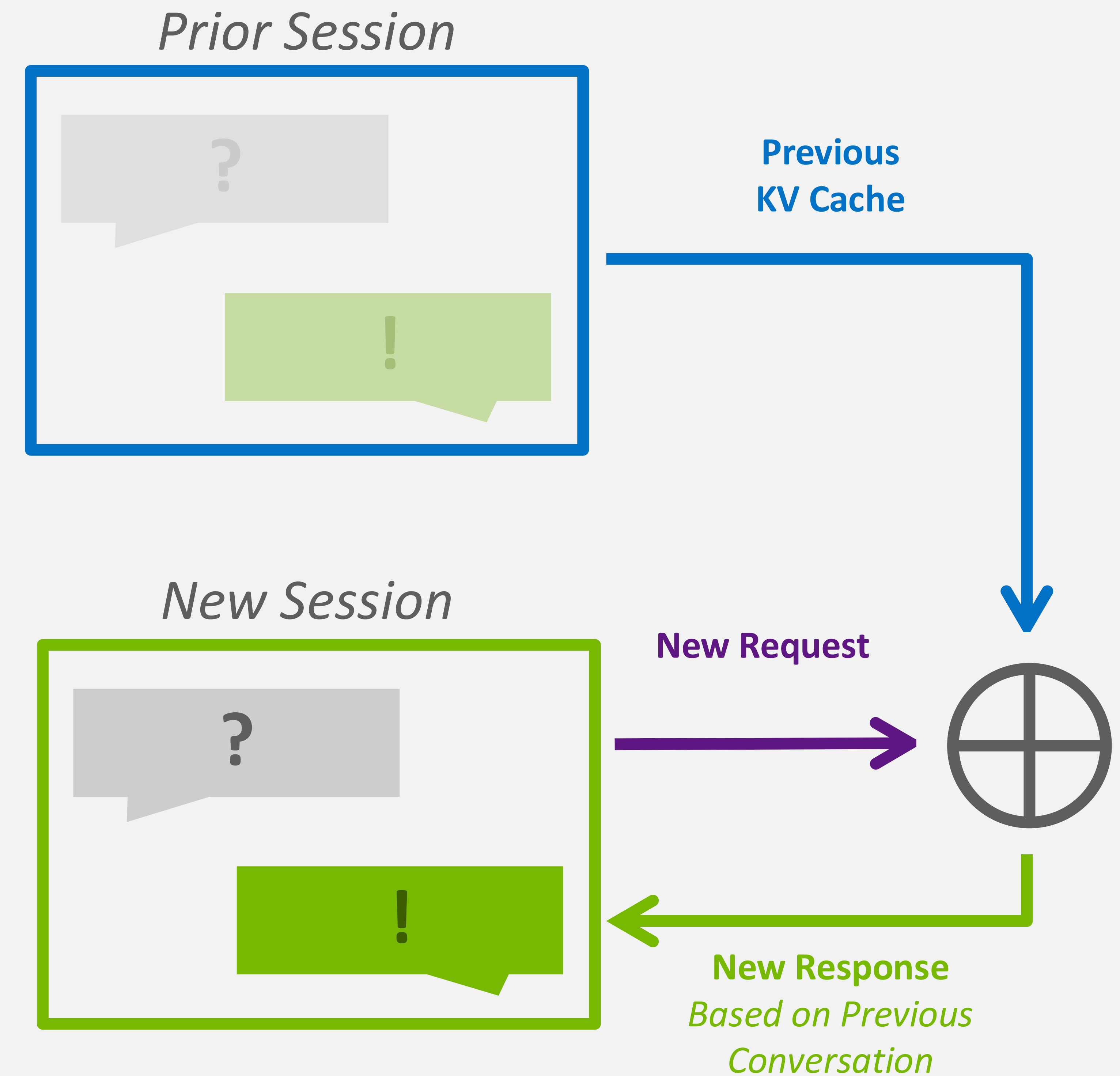


KV Cache Reusage

System Prompt Caching & Block reusage

Allows for interactive/ turn based systems & System Prompts

- Load prior KV cache blocks to avoid recomputation
 - Saves significant compute
 - Reduces Start-up time
- Block reuse allows for turn-based (chat) applications
 - Allows for additional options for intelligently reusing blocks
- System prompts allows for a preset KV cache for the LLM
 - E.g. to give rules, personality, or prior knowledge

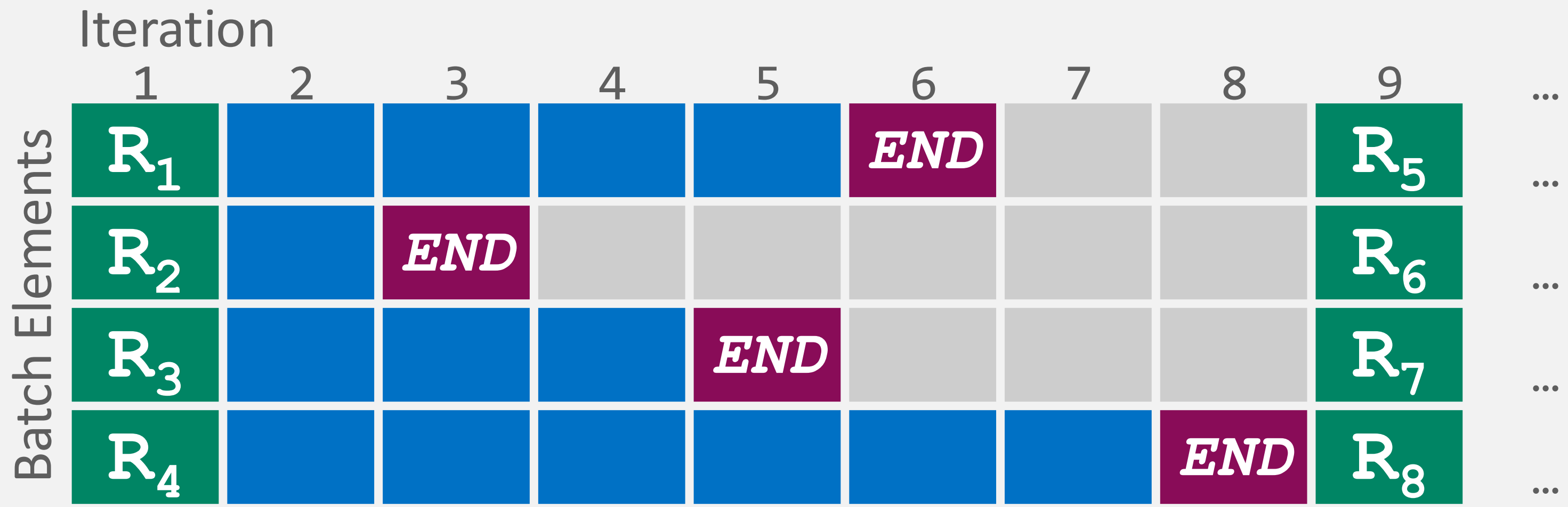


Inflight Batching

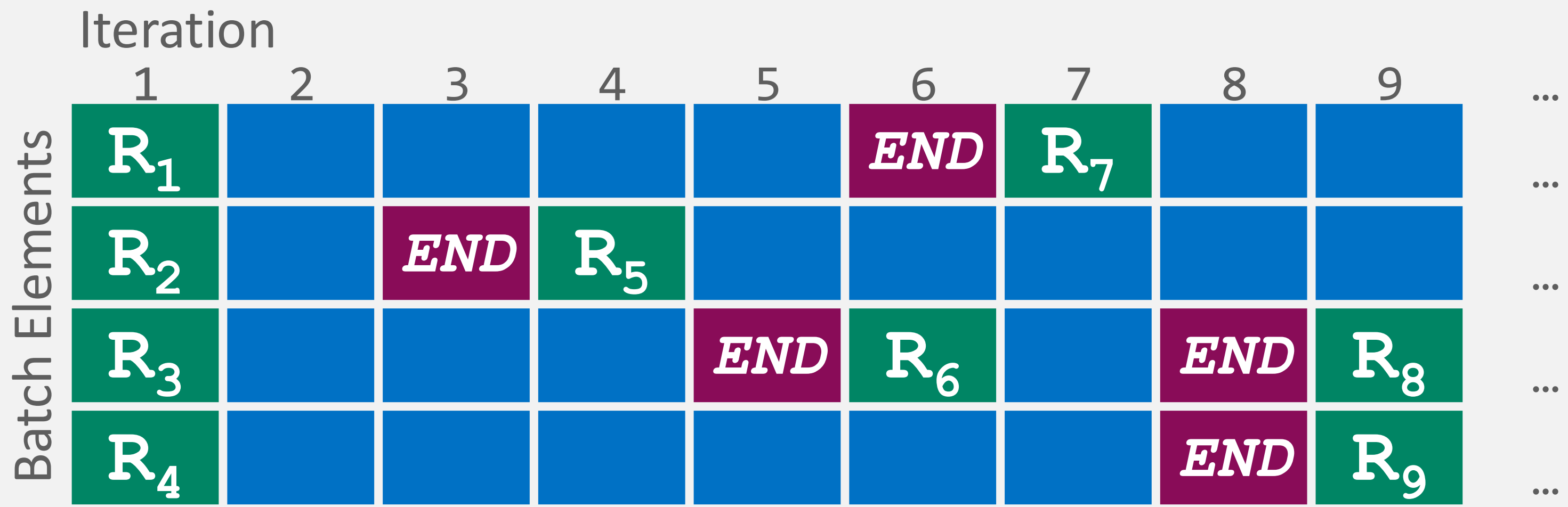
Maximing GPU Utilization during LLM Serving

TensorRT-LLM provides custom Inflight Batching to optimize GPU utilization during LLM Serving

- Replaces completed requests in the batch
 - Evicts requests after EoS & inserts a new request
- Improves throughput, time to first token, & GPU utilizaiton
- Integrated directly into the TensorRT-LLM Triton backend
- Accessible though the TensorRT-LLM Batch Manager



Static Batching



Inflight Batching

Context | Gen | EoS | NoOp

KV Cache Optimizations

Paged & Quantized KV Cache

Paged KV Cache improves memory consumption & utilization

- Stores keys & values in non-contiguous memory space
- Allows for reduced memory consumption of KV cache
- Allocates memory on demand

Quantized KV Cache improves memory consumption & perf

- Reduces KV Cache elements from 16b to 8b (or less!)
- Reduces memory transfer improving performance
- Supports INT8 / FP8 KV Caches

Both allow for increased peak performance

KV Cache Contents:
TensorRT-LLM optimizes inference on NVIDIA GPUs ...

Block 0	TensorRT	LLM	is	...
Block 1				
Block 2	Hello	World		
Block 3				

Traditional KV Caching

B ₀	TensorRT	LLM	is	...
B ₁				
B ₂	Hello	World		
B ₃				

Paged KV Cache

B ₀	TRT	LLM	is	...				
B ₁								
B ₂	Hello	World						
B ₃								

Quantized Paged KV Cache

Request 1	Request 2	Wasted	Free
-----------	-----------	--------	------

Quantization

How to Chose a Precision

- Best precision varies by application
 - FP8 activations generally provides best performacne
- Weight quantization reduces memory footprint & traffic
 - Reduces latency
 - Can fit larger models
 - Costs compute time to unpack the weights
- Activation quantization saves on compute
 - Improves throughput
 - Can run larger batch sizes
- WXAY = weights quantized to X bits, and activations to Y
- [Quantization Guide](#)

Method	Performance Improvement		Accuracy impact	Calibration time
	small batch BS <=4	large batch BS>=16		
FP8 (W8A8)	Medium	Medium	Very low / None	O(1min)
INT8 SQ (W8A8)	Medium	Medium	Medium	O(1min)
INT8 WO (W8A16)	Medium	None	Low	None
INT4 WO (W4A16)	High	None	High	None
INT4 AWQ (W4A16)	High	None	Low	O(10min)
INT4 GPTQ (W4A16)	High	None	Low	O(10min)
INT4-FP8 AWQ (W4A8)	High	Medium	Low	O(10min)

SQ = Smooth Quant
WO = Weight Only
AWQ = Activation Aware Quantization

Smooth Quant – Weight and Activation Quantization

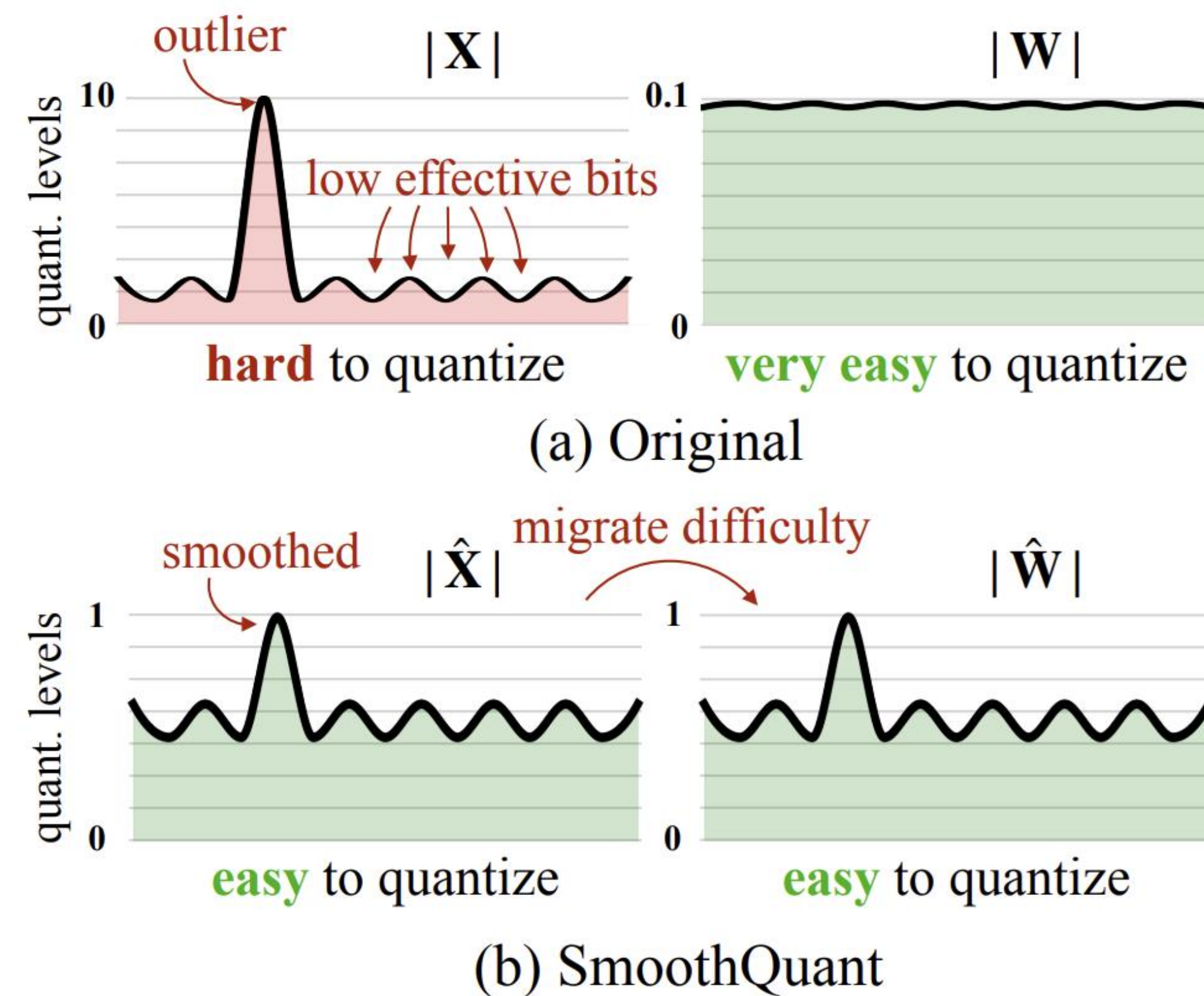


Figure 2: SmoothQuant’s intuition: the activation $\mathbf{X}$ is hard to quantize because outliers stretch the quantization range, leaving few effective bits for most values. We migrate the scale variance from activations to weights $\mathbf{W}$ during offline to reduce the quantization difficulty of activations. The smoothed activation $\hat{\mathbf{X}}$ and the adjusted weight $\hat{\mathbf{W}}$ are both easy to quantize.

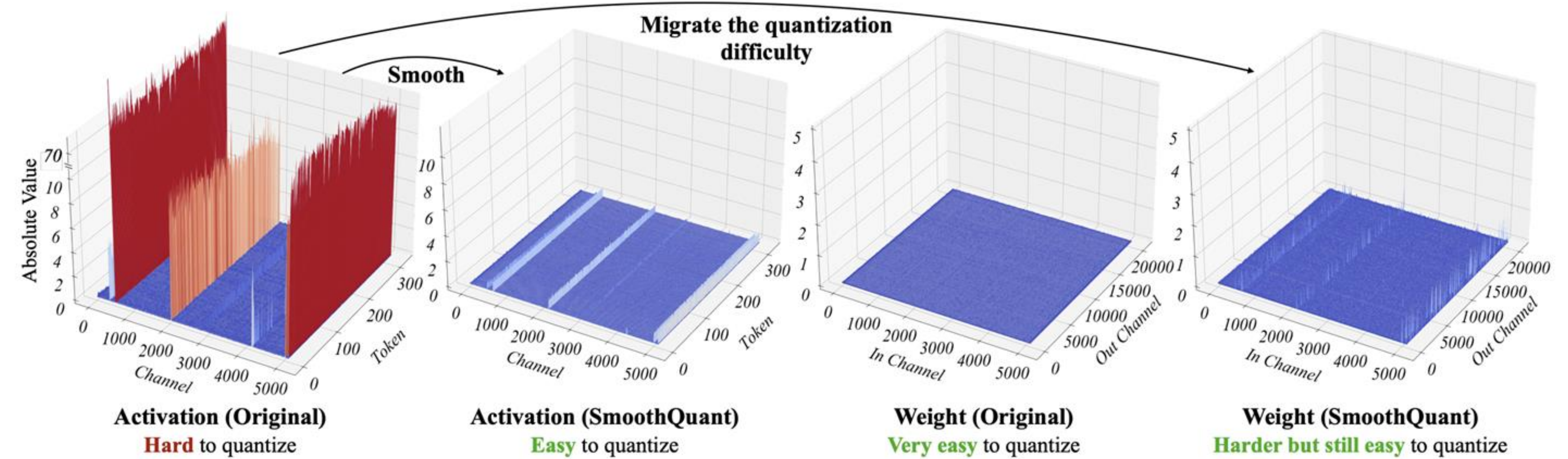


Figure 4: Magnitude of the input activations and weights of a linear layer in OPT-13B before and after SmoothQuant. Observations: (1) there are a few channels in the original activation map whose magnitudes are very large (greater than 70); (2) the variance in one activation channel is small; (3) the original weight distribution is flat and uniform. SmoothQuant migrates the outlier channels from activation to weight. In the end, the outliers in the activation are greatly smoothed while the weight is still pretty smooth and flat.

Smooth Quant – Weight and Activation Quantization

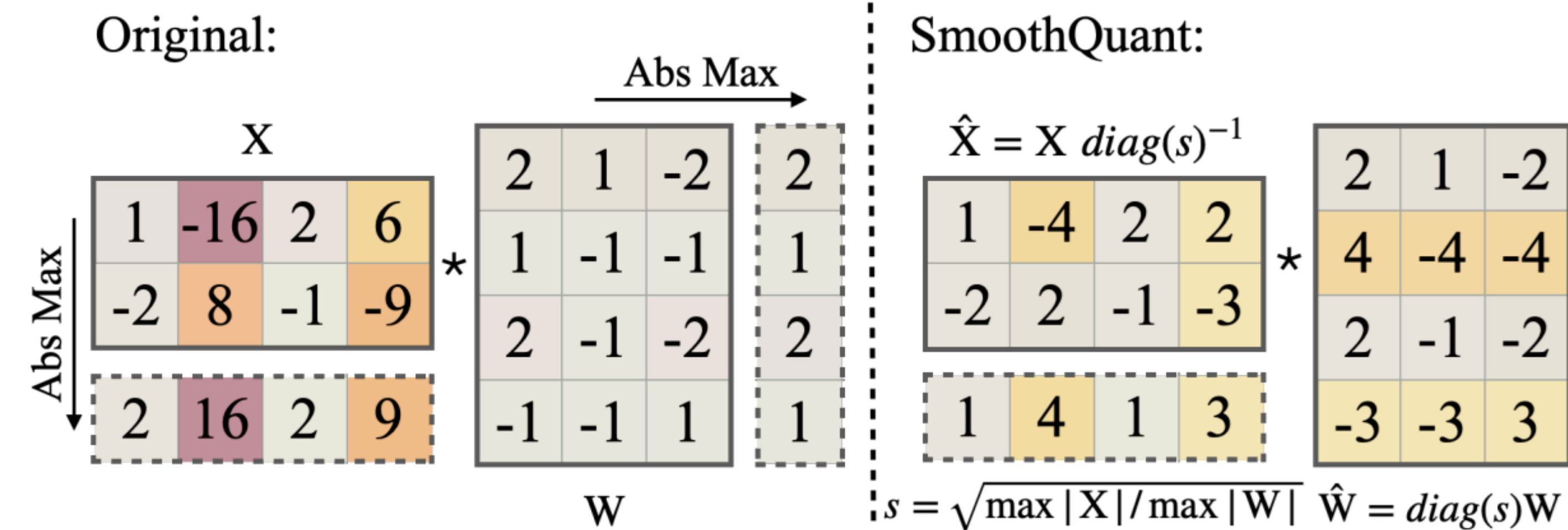


Figure 5: Main idea of SmoothQuant when α is 0.5. The smoothing factor s is obtained on calibration samples and the entire transformation is performed offline. At runtime, the activations are smooth without scaling.

Table 4: SmoothQuant works for different LLMs. We can quantize the 3 largest, openly available LLM models into INT8 without degrading the accuracy. For OPT-175B and BLOOM-176B, we show the average accuracy on WinoGrande, HellaSwag, PIQA, and LAMBADA. For GLM-130B we show the average accuracy on LAMBADA, MMLU, MNLI, and QNLI. *Accuracy is not column-wise comparable due to different datasets.

Method	OPT-175B	BLOOM-176B	GLM-130B*
FP16	71.6%	68.2%	73.8%
W8A8	32.3%	64.2%	26.9%
ZeroQuant	31.7%	67.4%	26.7%
LLM.int8()	71.4%	68.0%	73.8%
Outlier Suppression	31.7%	54.1%	63.5%
SmoothQuant-O1	71.2%	68.3%	73.7%
SmoothQuant-O2	71.1%	68.4%	72.5%
SmoothQuant-O3	71.1%	67.4%	72.8%

Smooth Quant – Weight and Activation Quantization

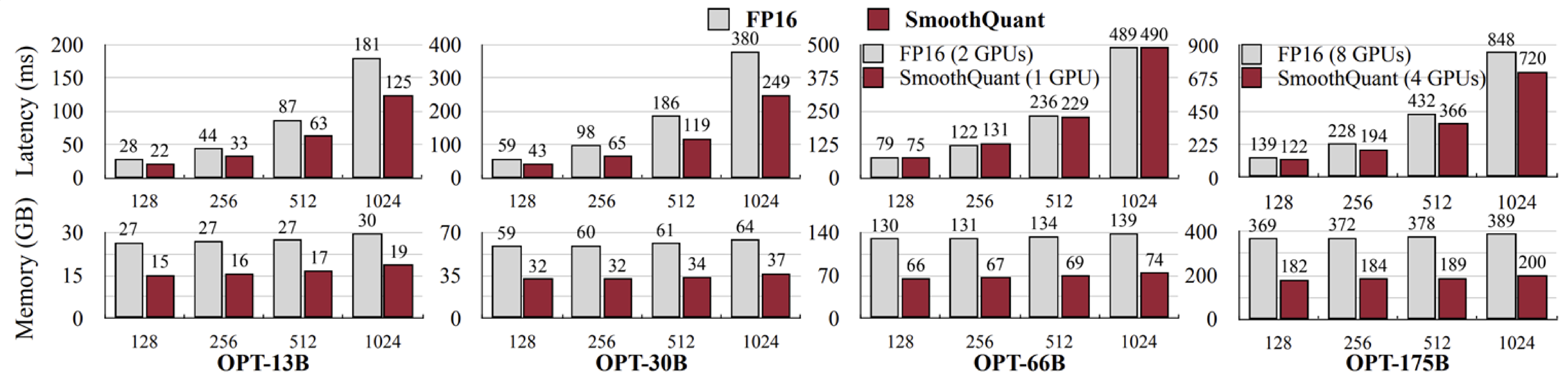


Figure 9: Inference latency (top) and memory usage (bottom) of the FasterTransformer implementation on NVIDIA A100-80GB GPUs. For smaller models, the latency can be significantly reduced with SmoothQuant-O3 by up to 1.56x compared to FP16. For the bigger models (OPT-66B and 175B), we can achieve similar or even faster inference using only **half** number of GPUs. Memory footprint is almost halved compared to FP16.

NVIDIA NIM

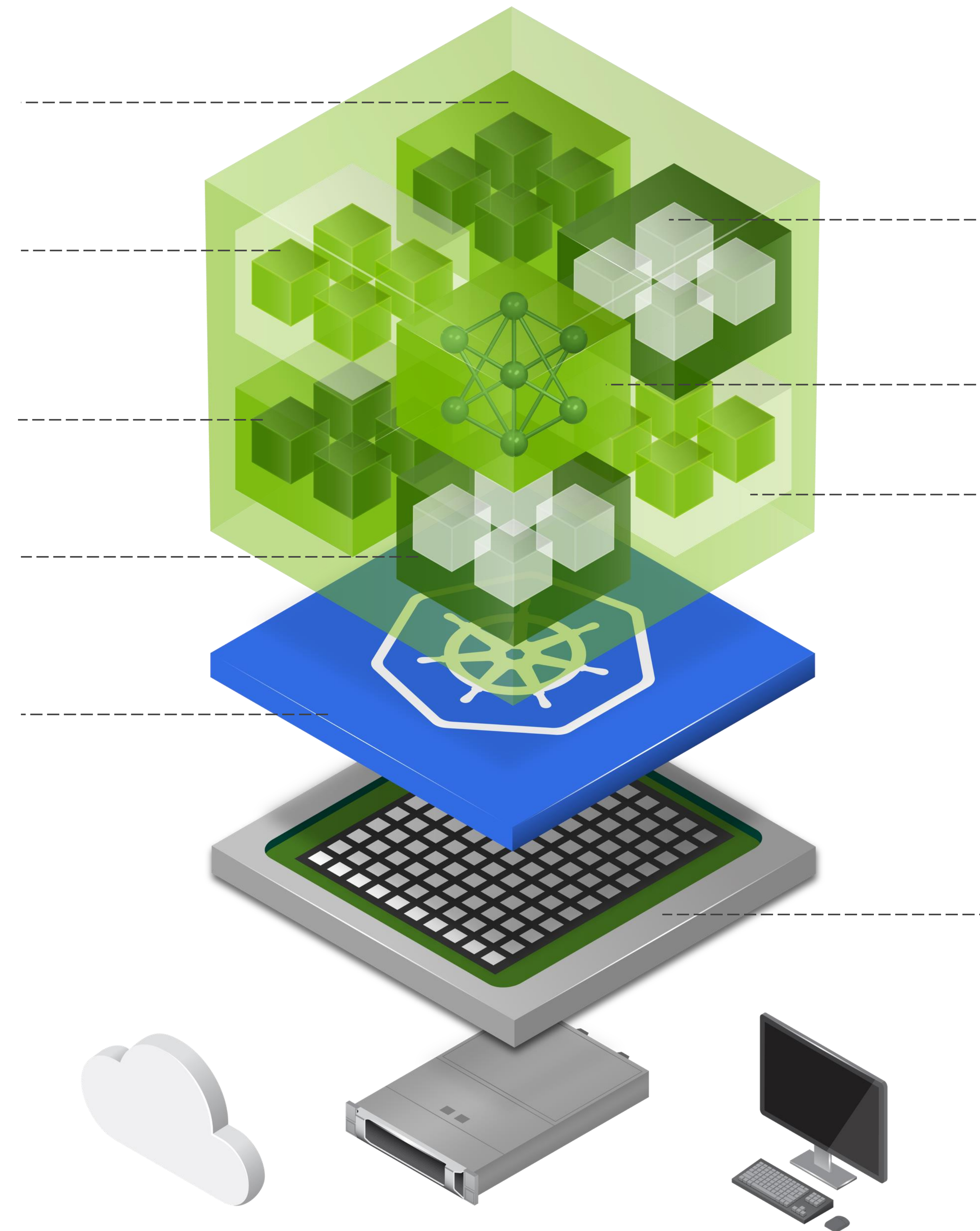
Standard APIs
Text, Speech, Image,
Video, 3D, Biology

NVIDIA Triton Inference Server
cuDF, CV-CUDA, DALI, NCCL,
Postprocessing Decoder

Cloud-Native Stack
GPU Operator, Network Operator

Enterprise Management
Health Check, Identity, Metrics,
Monitoring, Secrets Management

Kubernetes



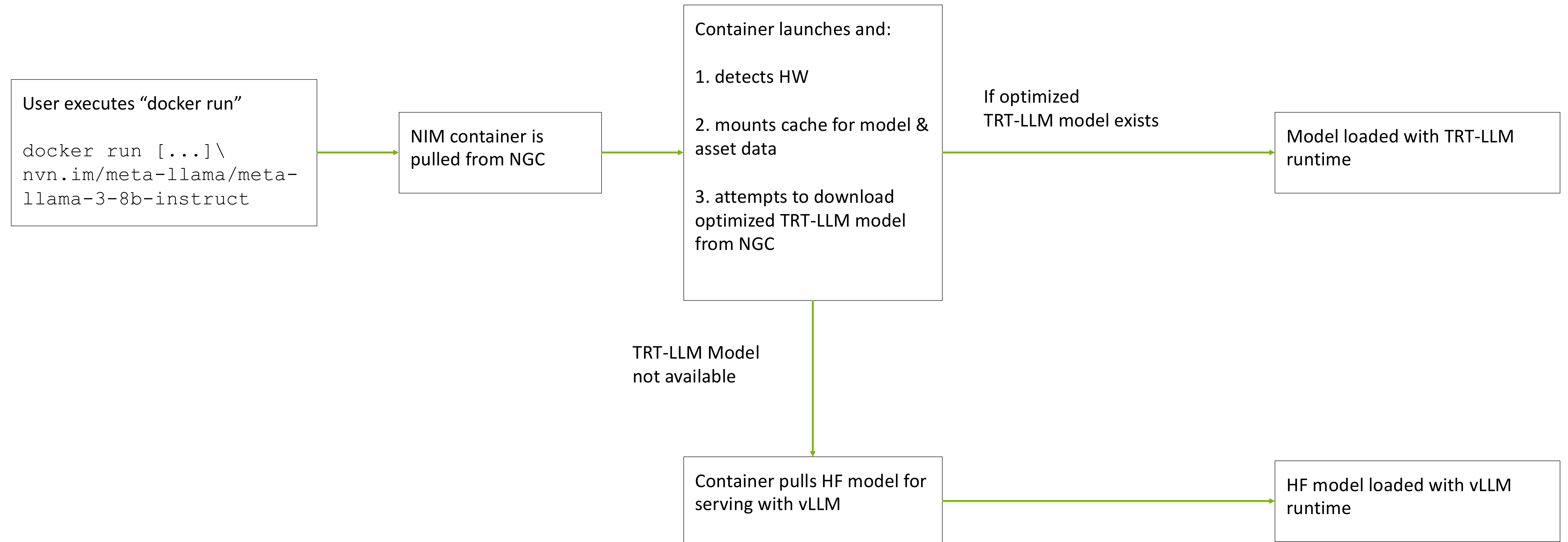
NVIDIA TensorRT and TensorRT-LLM
cuBLAS, cuDNN, In-Flight Batching,
Memory Optimization, FP8 Quantization

Optimized Model
Single GPU, Multi-GPU, Multi-Node

Customization Cache
P-Tuning, LoRA, Model Weights

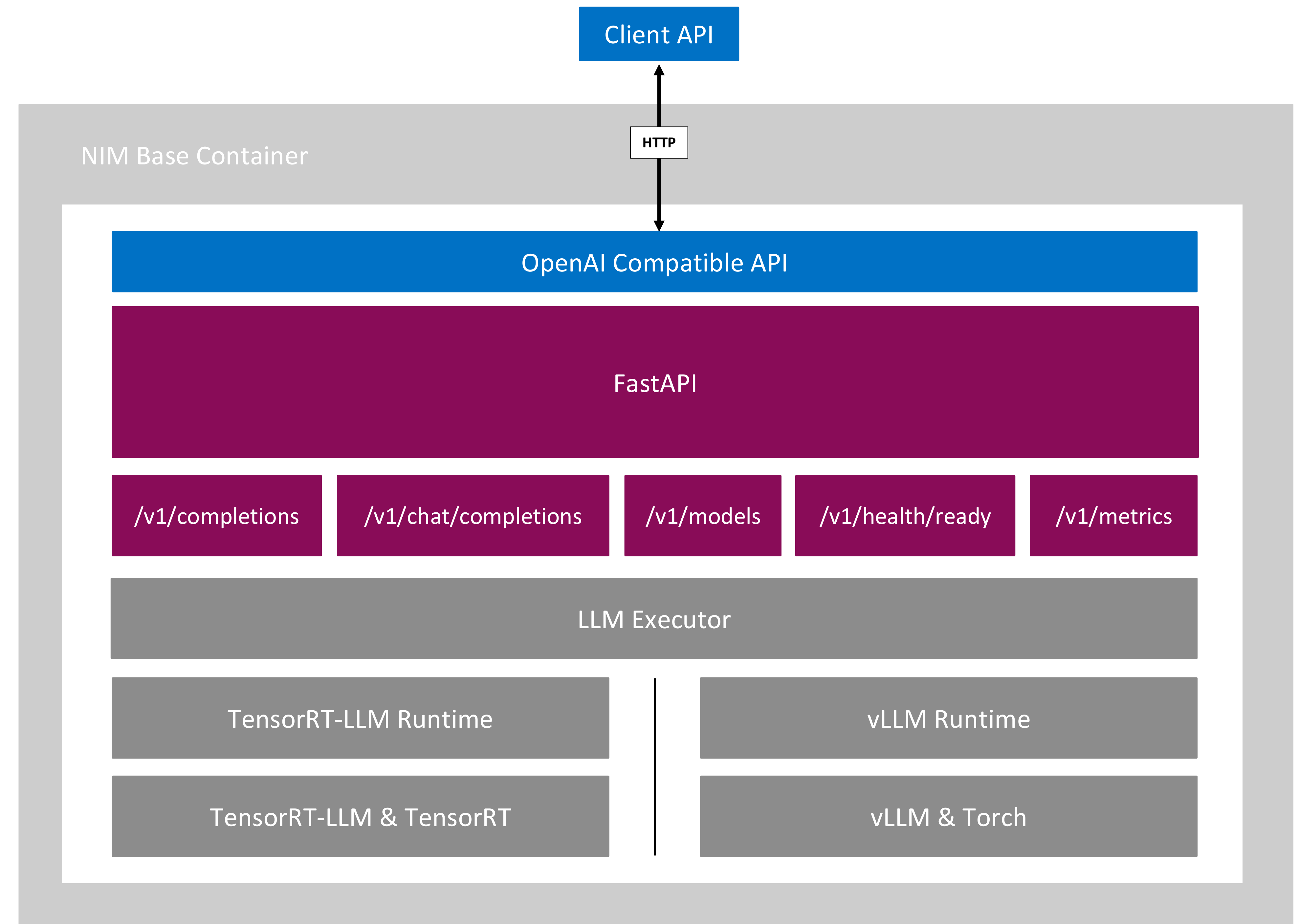
CUDA

NVIDIA LLM NIM Pull Sequence



NVIDIA NIM for LLM Architecture

- HTTP REST API conforms to OpenAI specification for easy developer integration
- Liveness, health check and metrics endpoints for monitoring and enterprise management
- NVIDIA NIM includes multiple LLM runtimes
 - TensorRT-LLM and vLLM
 - Runtime is selected based on detected hardware and available optimized engines, with preference given to optimized engines



Best Accuracy For Enterprise

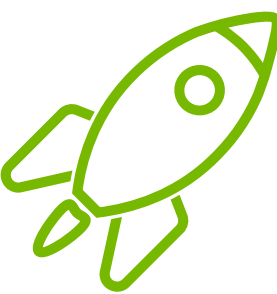
Seamlessly Deploy One Foundation Model Fine-Tuned for Multiple Tasks



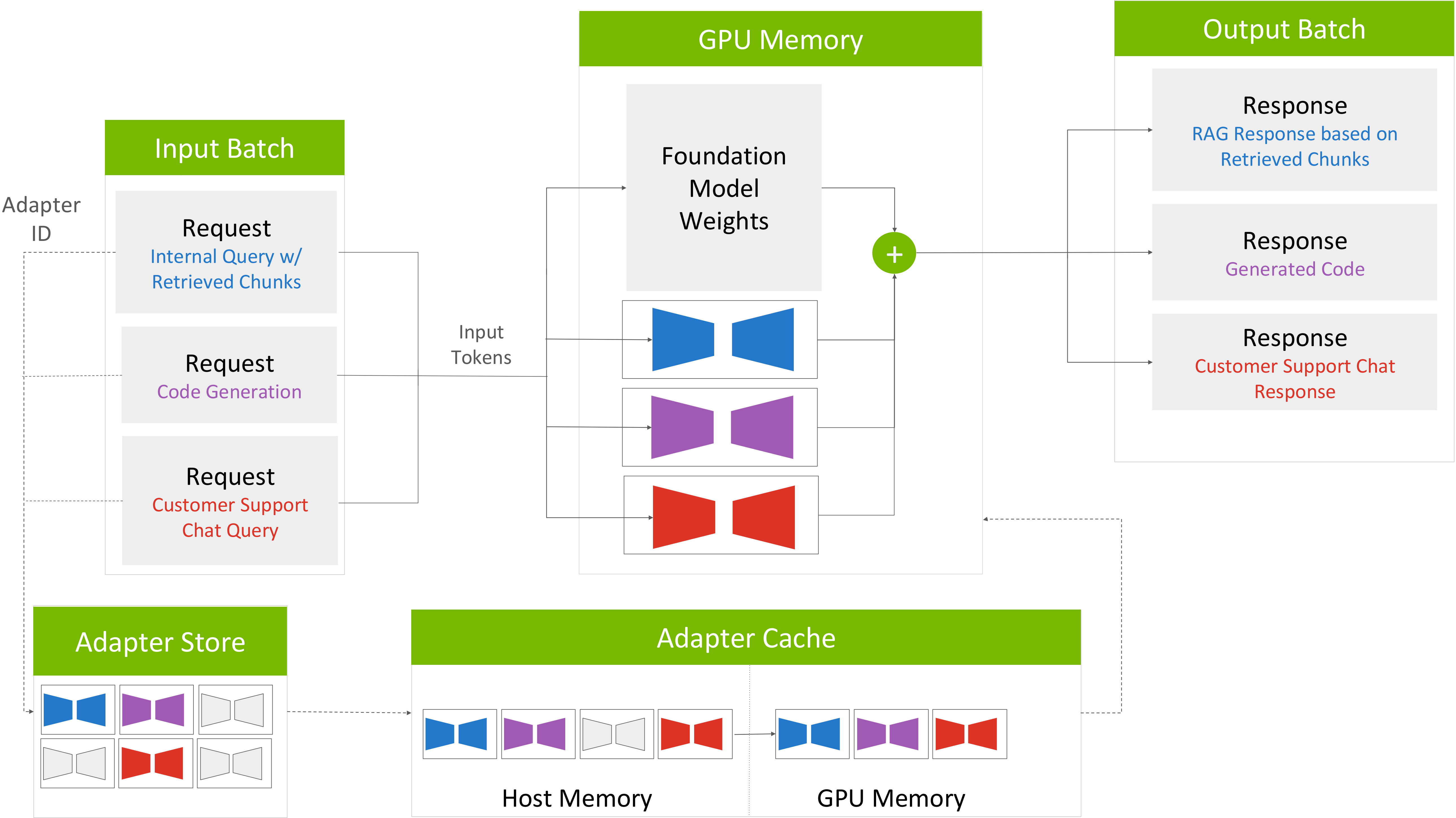
Best accuracy with NIMs customized for domain-specific tasks using NeMo Framework or Hugging Face PEFT



Optimization with custom CUDA kernels for simultaneous multi-LoRA and base model inference, plus automatic multi-tier cache management

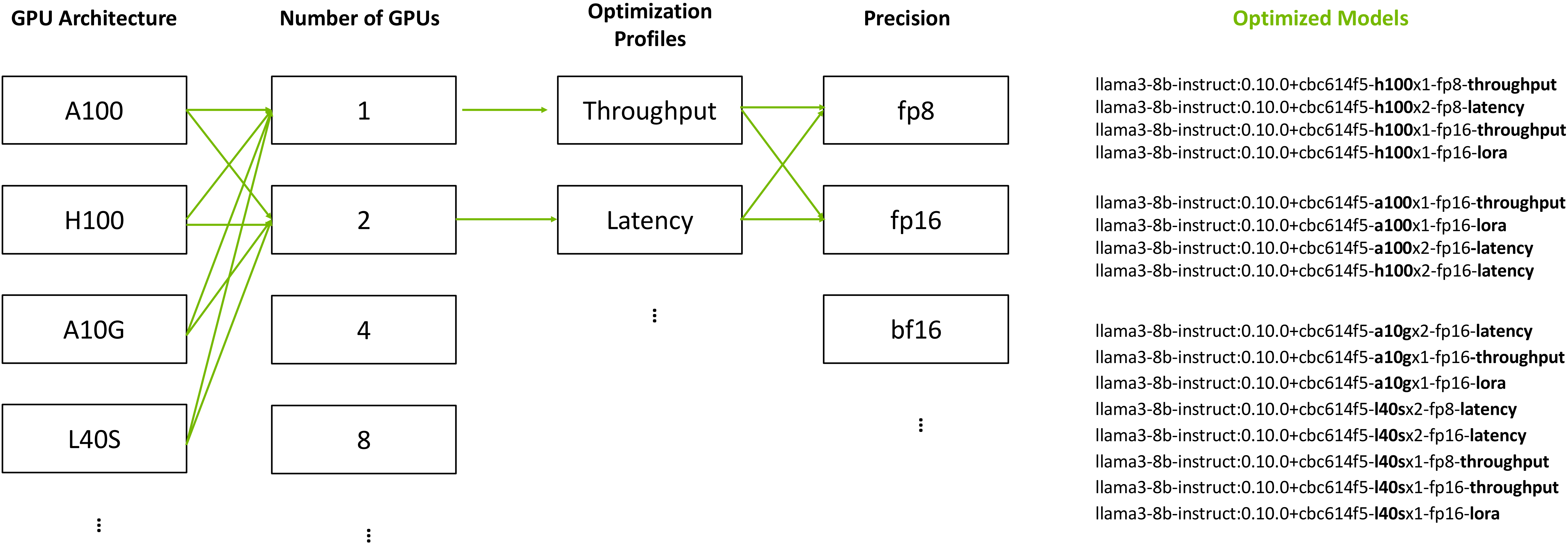


Seamless deployment with a single command for both base and LoRA models



How NIMs Leverage Optimized Engines

Llama-3-8B-Instruct Optimization for NVIDIA GPUs with LoRA Support Option



Optimized Models

llama3-8b-instruct:0.10.0+cbc614f5-**h100**x1-fp8-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x2-fp8-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x1-fp16-**lora**

llama3-8b-instruct:0.10.0+cbc614f5-**a100**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**a100**x1-fp16-**lora**
llama3-8b-instruct:0.10.0+cbc614f5-**a100**x2-fp16-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x2-fp16-**latency**

llama3-8b-instruct:0.10.0+cbc614f5-**a10g**x2-fp16-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**a10g**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**a10g**x1-fp16-**lora**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x2-fp8-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x2-fp16-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x1-fp8-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x1-fp16-**lora**

```
Detected 3 compatible profiles.  
Valid profile: 17190a87573ad181e4a55b96d1a84b52f1c82c6542e2404473b25011ebfb403e (tensorrt_llm-A100-bf16-tp1-balanced)  
Valid profile: 4ca08b7e7f1d59d5da5f761969320738d6922f25f26f334ab344c21f2e57348f (tensorrt_llm-A100-fp16-tp1-throughput)  
Valid profile: 15fc17f889b55aedaccb1869bfeadd6cb540ab26a36c4a7056d0f7a983bb114f (vllm)  
Selected profile: 17190a87573ad181e4a55b96d1a84b52f1c82c6542e2404473b25011ebfb403e (tensorrt_llm-A100-bf16-tp1-balanced)  
Profile metadata: llm_engine: tensorrt-llm
```

Learn How TRT-LLM Improves TCO

By reducing memory and increasing speed of inference, more requests can be processed for same compute

Speed up inference with SOTA quantization techniques in TRT-LLM

The deployment and inference speed of LLMs are often impeded by limitations in memory capacity, memory bandwidth, and computation power. Quantization emerges as a vital strategy to address these bottlenecks, involving representing weights and activations with lower-precision data types like [FP8](#).

In this blog, we provide an overview of the quantization features in TensorRT-LLM, share benchmark, and offer best practices of selecting the appropriate quantization methods tailored to your specific use case.

Quantization in TensorRT-LLM

TensorRT-LLM offers a best-in-class unified quantization toolkit to significantly speedup DL/GenAI deployment on NVIDIA hardware, while maintaining model accuracy. This toolkit is designed with easy-of-use in mind. You can follow [this user guide](#) to quantize [supported LLMs](#) with a few lines of codes. We currently focus on providing SOTA **Post-Training Quantization (PTQ)** and will soon expand to more model optimization techniques in the near future.

Benchmark

Performance

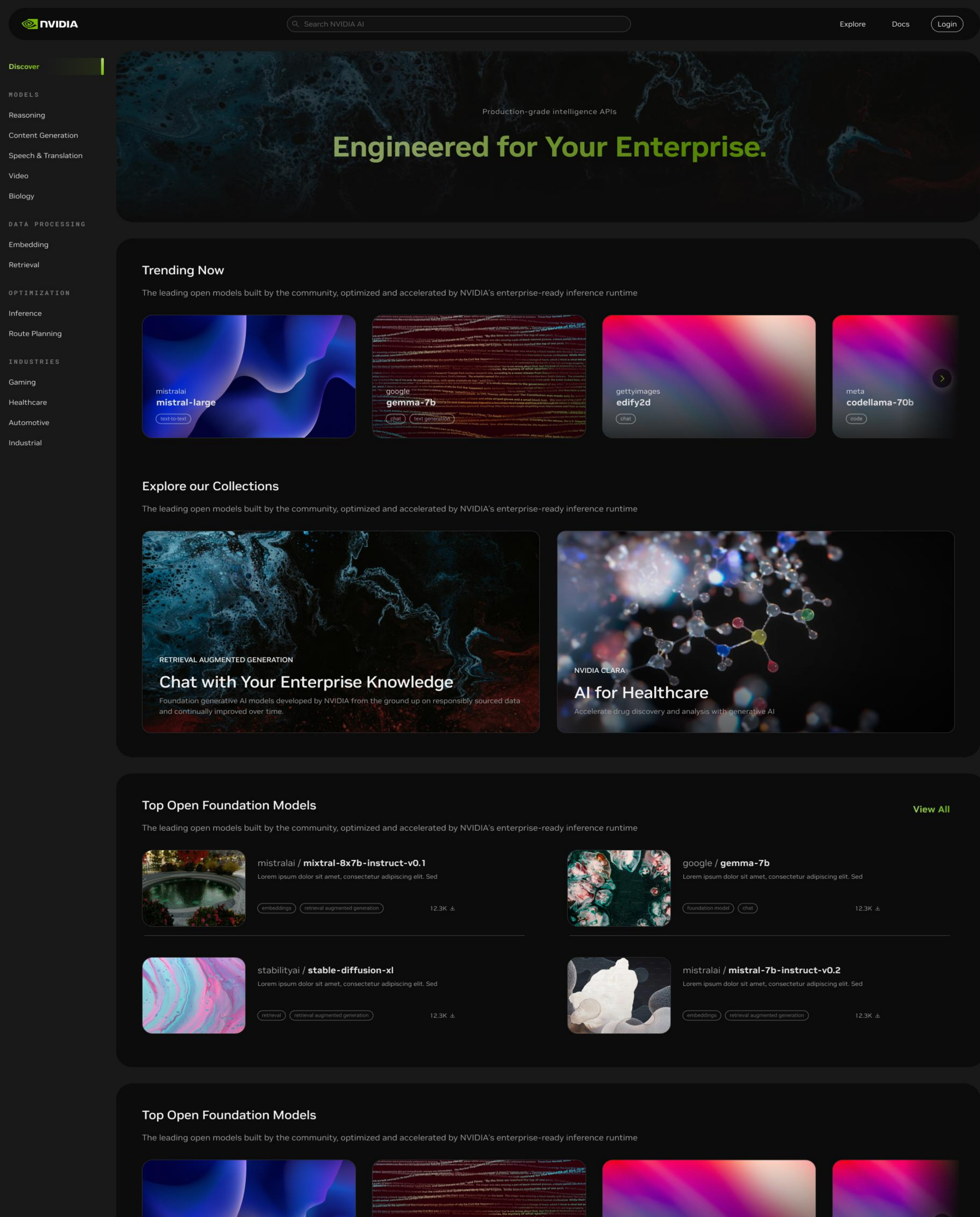
In the following benchmark, we highlight the acceleration of a few popular models at a small batch size without imposing latency constraints. It's important to note that in scenarios where there's a latency constraint in your application, TRT-LLM can achieve an even greater performance improvement. Using LLaMA-v2-7B as an example, when the first token latency is constrained to be under 500ms, quantization with FP8 and a batch size of 16 achieves a notable **2.3x inference speedup** compared to FP16 on a H100.

Model	Batch Size	Speedup (FP8 v.s. FP16)	Speedup (INT8 SQ v.s. FP16)
GPT-J	1	1.40x	1.40x
GPT-J	8	1.44x	1.30x
LLaMA-v2-7B	1	1.51x	1.47x
LLaMA-v2-7B	8	1.40x	1.32x

*The above benchmarks were run with Input Length=1024, Output Length=128, and TP=1 on H100 80GB.

Performance

		GPU	H200 141GB HBM3	GH200 120GB	H100 80GB HBM3	H100 80GB HBM3	A100- SXM4- 80GB	L40S
		Precision	FP8	FP8	FP8	Mixed	Mixed	FP8
Model	Input/Output Lengths	TP						
LLaMA v3 8B	128/128	1	28288.75	25420.52	27399.75	15567.44	6586.88	8745.80
	128/2048	1	23230.62	16426.68	19198.73	8817.39	4882.13	5084.49
	128/4096	1	16144.44	9832.66	12084.97	5352.37	3079.90	2755.13
	2048/128	1	3623.79	3290.22	3463.26	1852.48	781.63	980.86
	2048/2048	1	11093.62	7573.35	8894.11	3986.83	2268.13	2051.79
Mistral 7B	128/128	1	30223.01	27696.90	29788.46	16319.25	6807.02	9612.58
	128/2048	1	24989.54	17942.29	20509.72	9982.01	5296.02	5444.89
	128/4096	1	17036.14	10846.03	12807.80	5718.89	3241.33	2931.17
	2048/128	1	3678.80	3294.02	3521.71	1887.75	786.43	1002.49
	2048/2048	1	11510.54	8357.75	9214.61	4284.82	2363.25	2154.26



1000 Free Inference Requests

- Experience **optimized** foundation models
- Start prototyping for **free**
- Deploy anywhere
- Get access to NVIDIA AI Enterprise 90-day eval

build.nvidia.com



LinkedIn - Mark Moyou, PhD
mmoyou@nvidia.com